

ITCS212 Web Programming Project Report

Entertainment: MovFlix Web Application

Pongpop Lapvikai	6388028
Kittapon Lerdpayab	6388029
Paveena Kumnerdpun	6388088
Kanin Prakaikowit	6388099

Section 2 Group 5
Lect. Dr. Jidapa Kraisangka
Lect. Dr. Wudhichart Sawangphol
Faculty of Information and Communication Technology
Mahidol University

Table of Contents

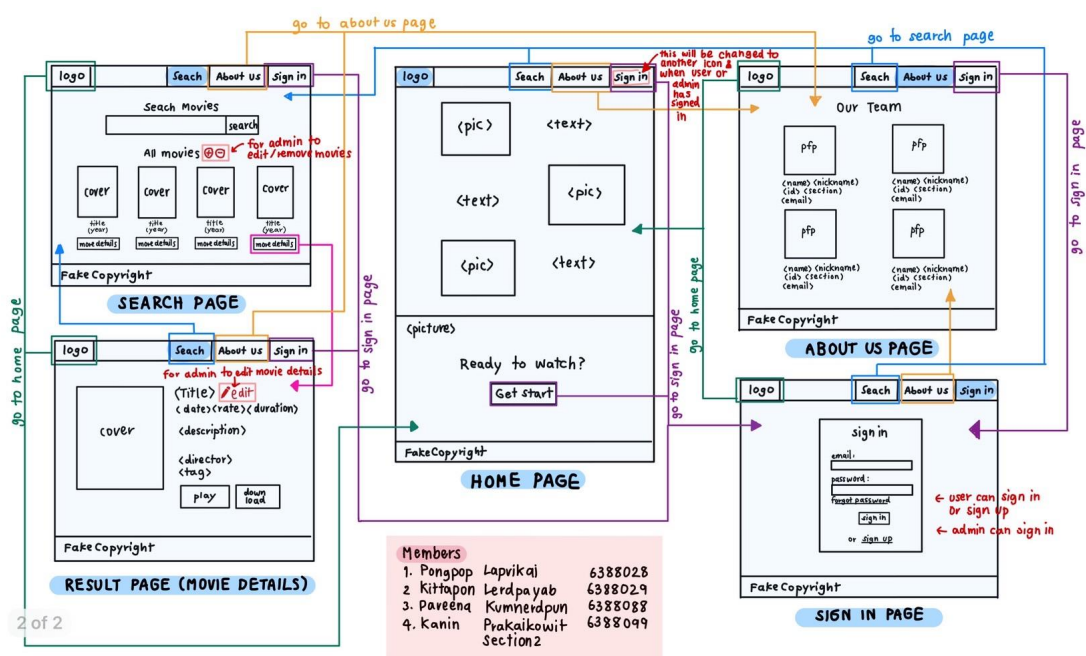
Overview	3
Navigation Diagram	3
Normal User Pages	4
Administrator Pages	15
Web Services	24
Web Services Testing Results	33

The Overview of MovFlix

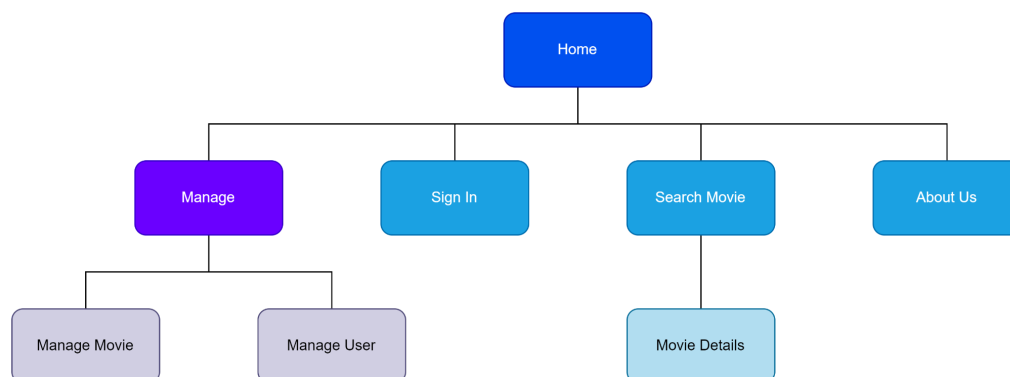
This report is intended to study the "Entertainment" and implement web application of this business domain. Our group was interested in the movie domain. Therefore, we develop a web application called MovFlix. MovFlix is a web application that allows users to stream movies and download movies to watch anywhere and anytime. It provides several web services for users such as authentication service, search movies service, user management service, and movie management service.

Navigation Diagram

Phase I & II



Phase III

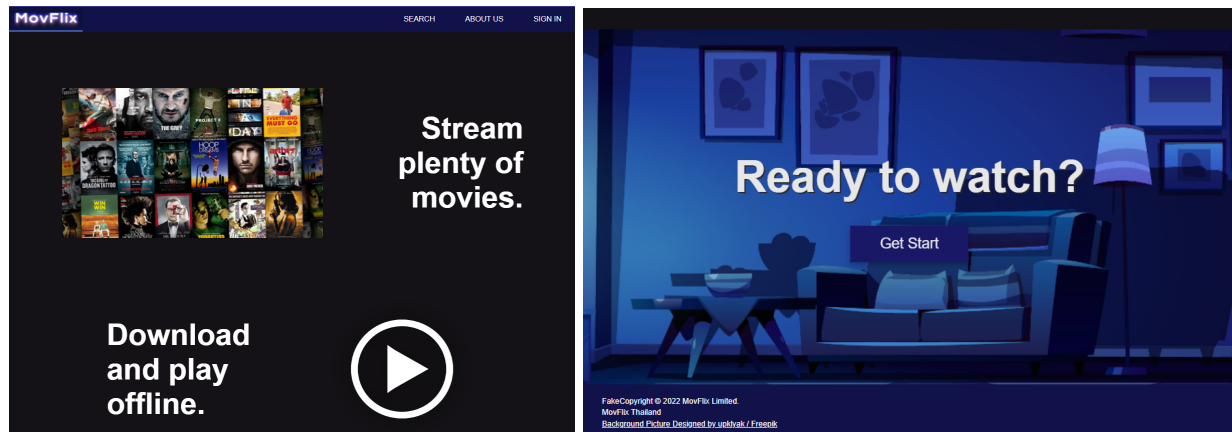


Normal User Pages

1. Home Page

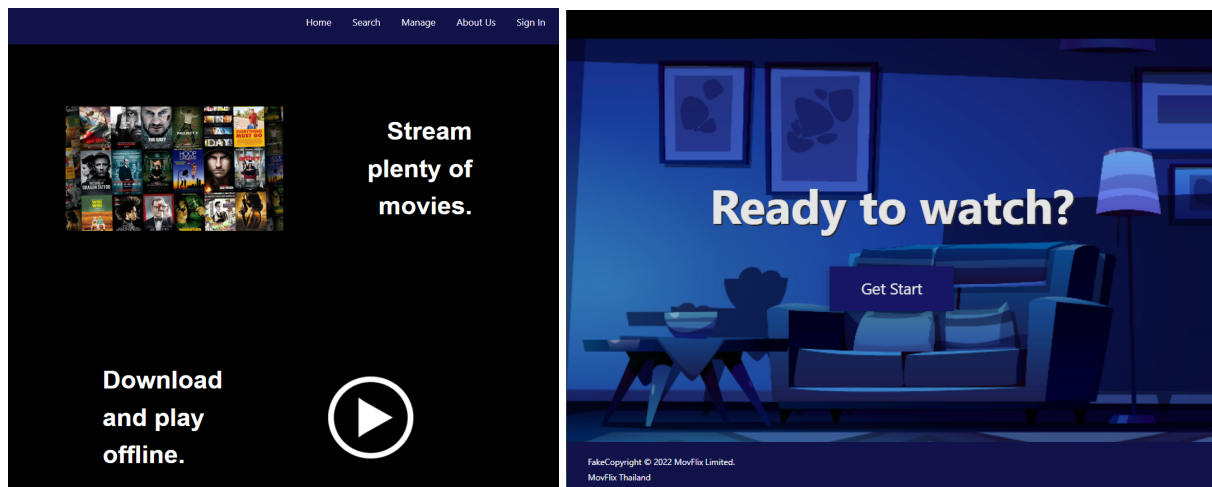
Home page shows the pictures and message that briefly describes the MovFlix website. When the user scrolls down to the bottom of the home page, there is a button “Get Start” which links to the sign in page.

Phase I & Phase II



This is the first design of the home page in phase I and it continues to be used in phase II.

Phase III: <http://localhost:3000/>



2. Sign In Page (for normal user)

Phase I & Phase II

MovFlix SEARCH ABOUT US SIGN IN

Sign in

Email
Type your email

Password
Type your password

[Forgot password?](#)

[Sign In](#)

Or Sign Up using
[SIGN UP](#)

FakeCopyright © 2022 MovFlix Limited.
MovFlix Thailand

This is the first design of the sign in page in phase I and it continue to be used in phase II.



In Phase II, normal users can sign in to MovFlix web application using email and password. Authentication service was implemented. Once the user is authenticated, it will show a link to the home page.



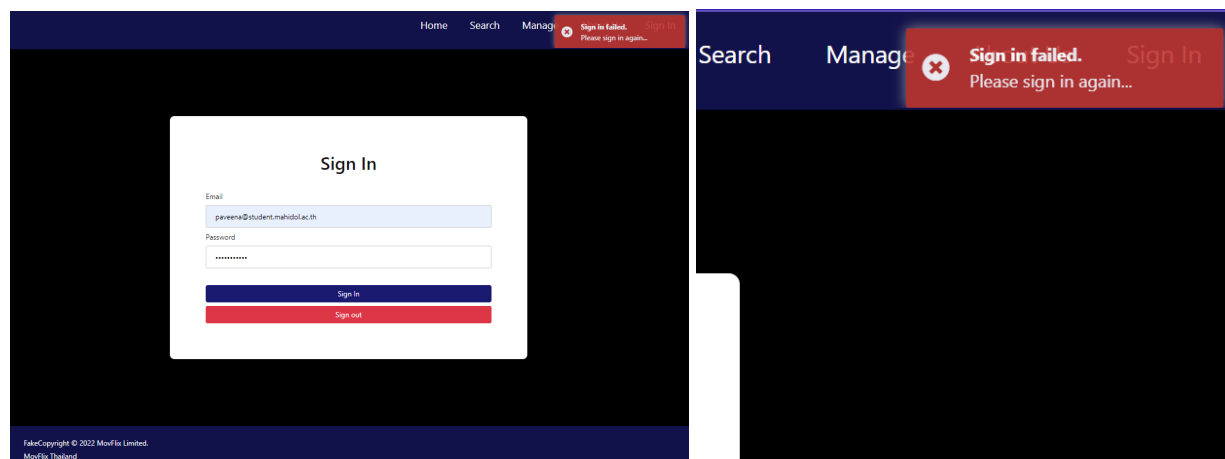
However, if the user is not authenticated, it will show a link to the sign in page for the user to sign in again.

Phase III: <http://localhost:3000/signin>

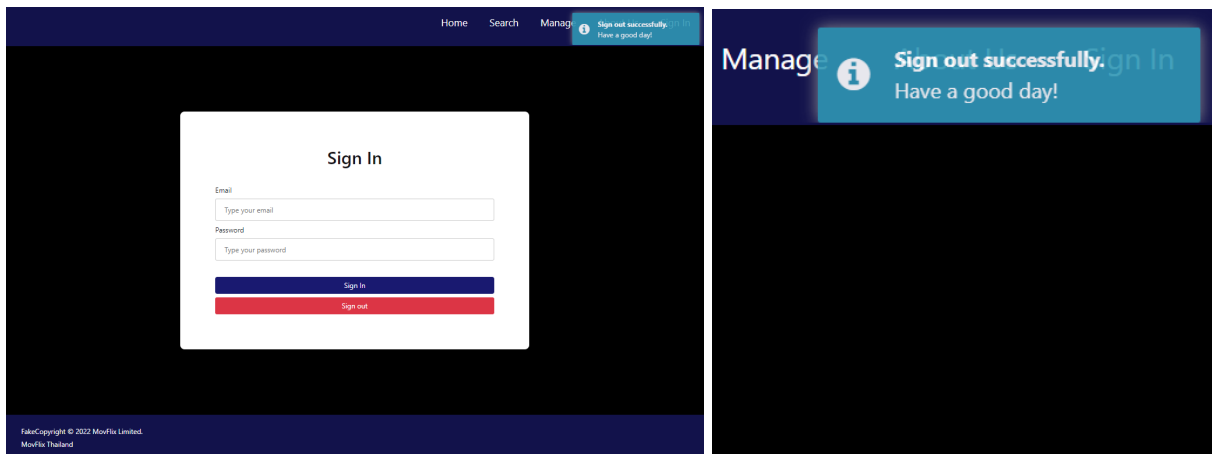
The screenshot shows the Sign In page of the MovFlix web application. The page has a dark blue header with navigation links: Home, Search, Manage, About Us, and Sign In. The main content area is black, and the Sign In form is a white card in the center. The form has two input fields: Email (containing 'paveena@student.mahidol.ac.th') and Password (containing '*****'). Below the fields are two buttons: a dark blue 'Sign In' button and a red 'Sign out' button. The footer is dark blue with the text 'FakeCopyright © 2022 MovFlix Limited. MovFlix Thailand'.

In phase III, normal users can sign in to MovFlix web application using email and password. Sign in page has one input box to insert email and another input box for inserting password. When the user inserts email and password and clicks the “Sign in” button, once the user is authenticated, it will redirect to the home page automatically.

However, if the user is not authenticated, it will notify the user to sign in again.



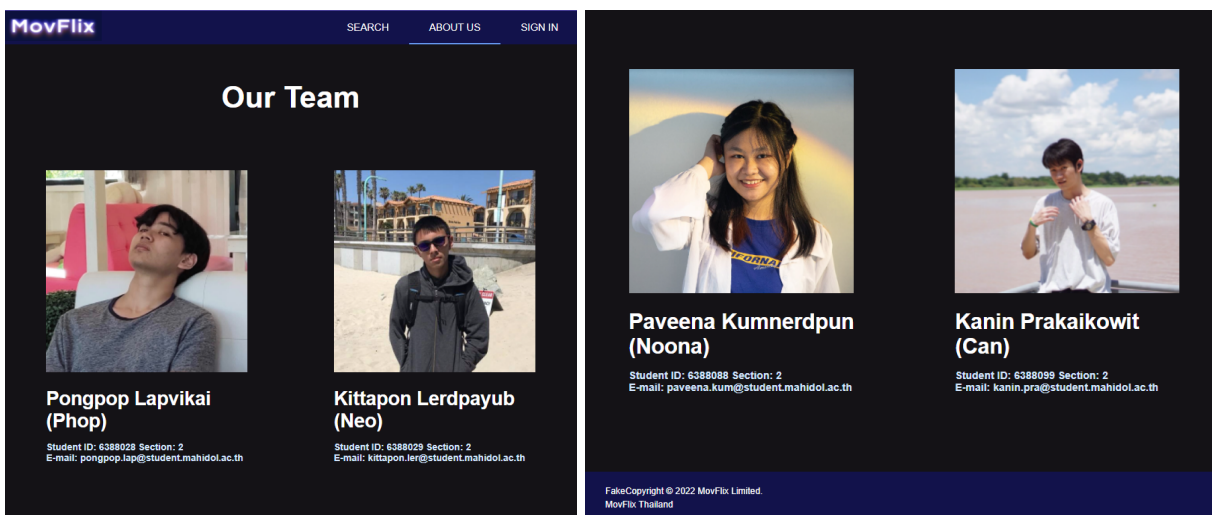
Once the user signs out, it will notify the user that signing out is successful.



3. About Us Page

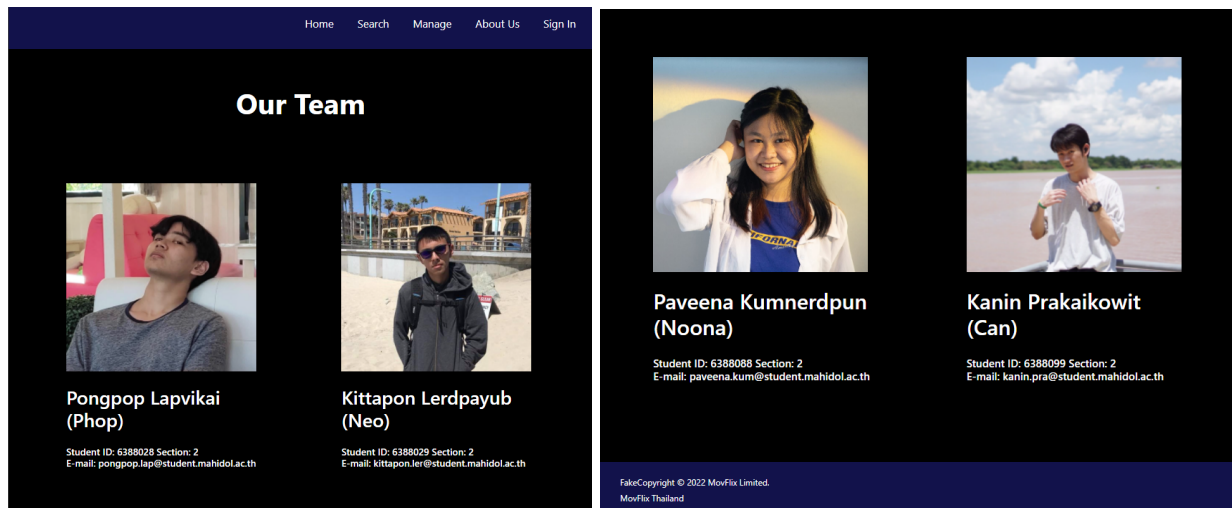
About us page shows the information of the developer team such as first name, last name, nickname, student ID, section and email.

Phase I & Phase II



This is the first design of about page in phase I and it continues to be used in phase II and phase III.

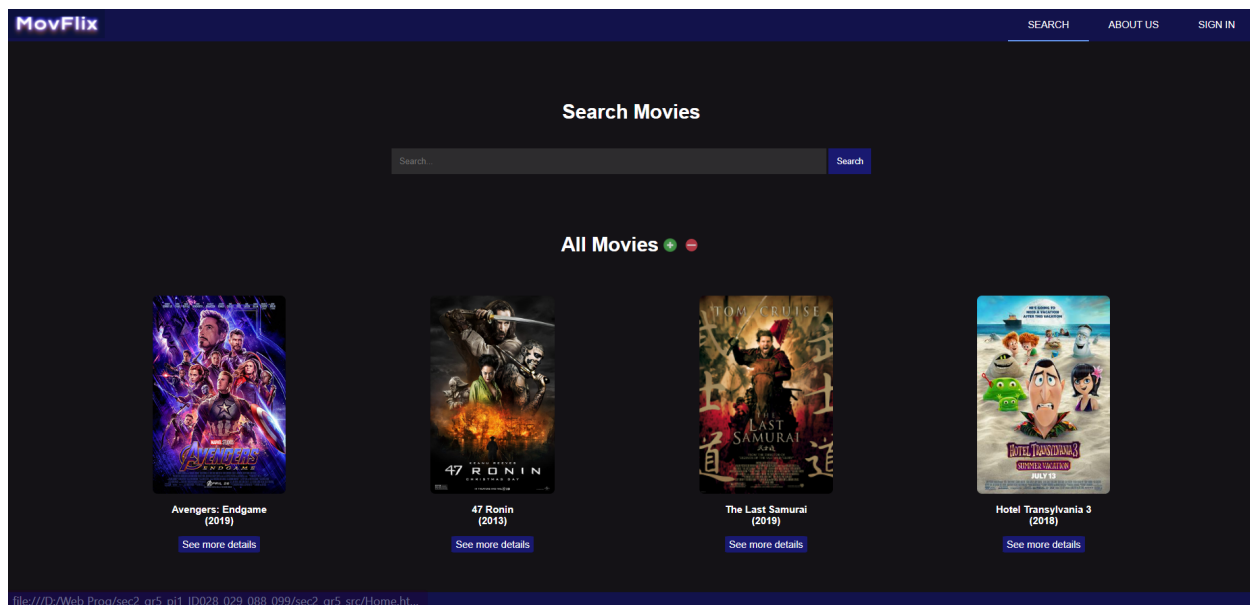
Phase III: <http://localhost:3000/aboutus>



4. Search Movie Page

Search page is for normal user to search movies that they would like to stream in MovFlix website.

Phase I

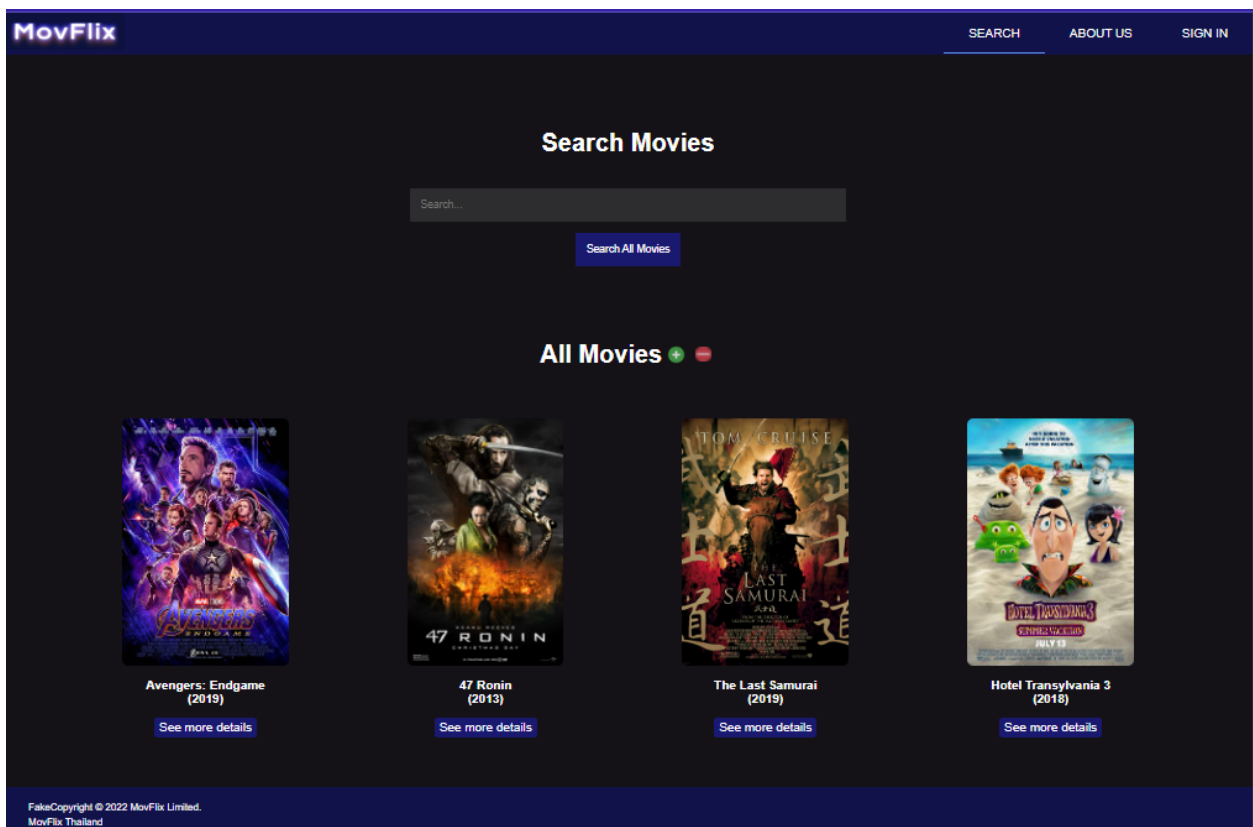


This is the first design of the search movie page in phase I.

Phase II



In Phase II, the authentication service is used. If the user has not signed in yet. It will show the user a link to the sign in page.



Once the users sign in, they can search the movies. However, the users can search only with no criteria, meaning that when they click on the “Search All Movies” button, it will show all movies provided in the web application.

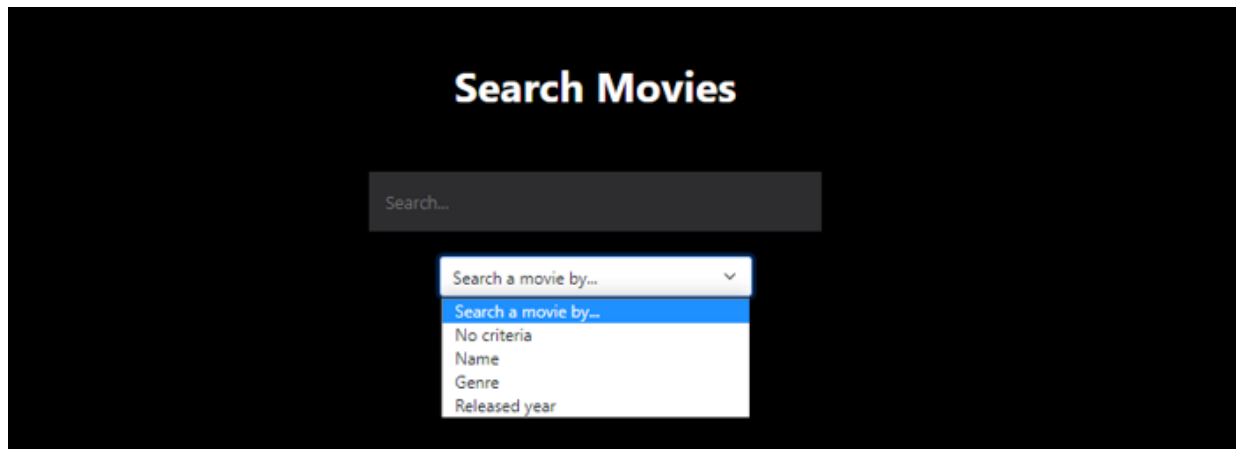
- [Avengers: Endgame](#)
Runtime 03:01:00
- [47 Ronin](#)
Runtime 02:08:00
- [The Last Samurai](#)
Runtime 02:34:00
- [Hotel Transylvania 3: Summer Vacation](#)
Runtime 01:37:00
- [Zootopia](#)
Runtime 01:48:00
- [The silence](#)
Runtime 01:30:00
- [A WHISKER AWAY](#)
Runtime 01:45:00
- [Godzilla: King of the Monsters](#)
Runtime 02:11:00
- [Ghost Wife](#)
Runtime 01:40:00

When they click on the “Search All Movies” button, it will show all movies provided in the web application, which the user can click on the movie name and it will go to the movie details page.

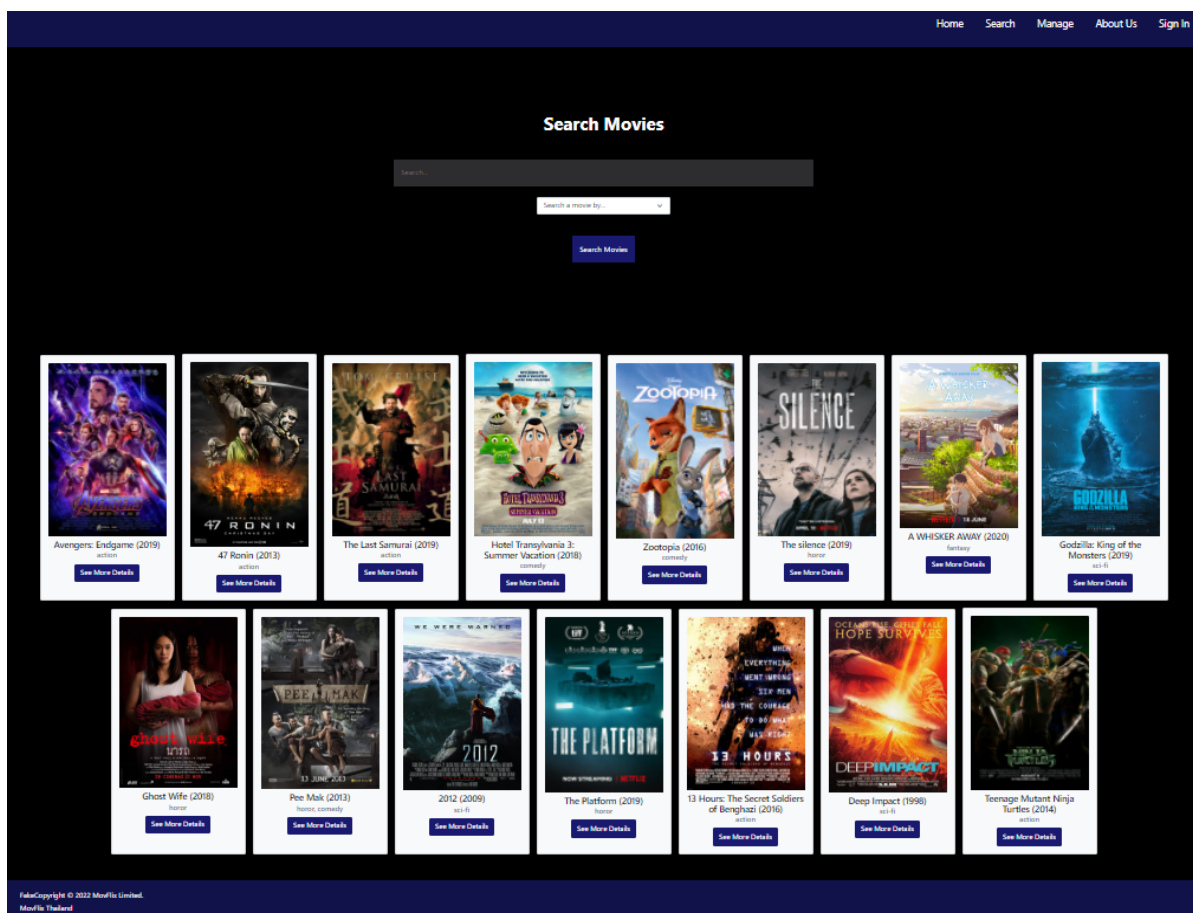
Phase III: <http://localhost:3000/search>

The screenshot shows a web application interface for searching movies. At the top, there is a dark blue navigation bar with links: Home, Search, Manage, About Us, and Sign In. The main content area has a dark background with the title 'Search Movies' in white. Below the title is a search input field with the placeholder text 'Search...'. Underneath the input field is a dropdown menu with the text 'Search a movie by...' and a downward arrow. Below the dropdown is a blue button with the text 'Search Movies'. At the bottom of the page, there is a dark blue footer with the text 'FakeCopyright © 2022 MovFlix Limited. MovFlix Thailand'.

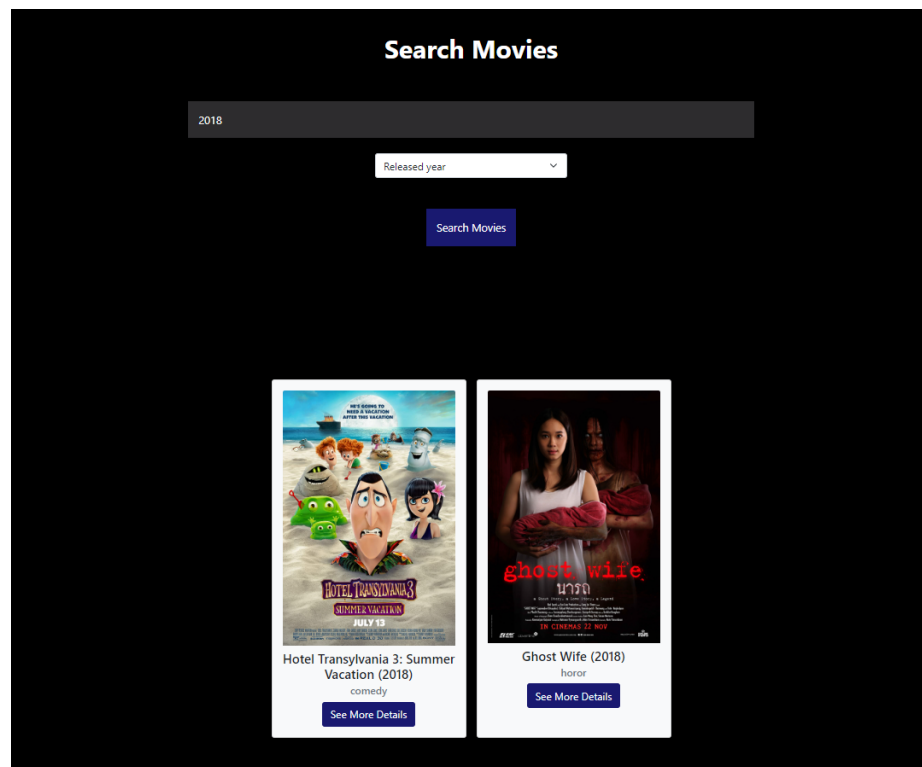
In phase III, users must sign in first before accessing the search movie page. If the user has not signed in, it will redirect to the sign in page automatically.



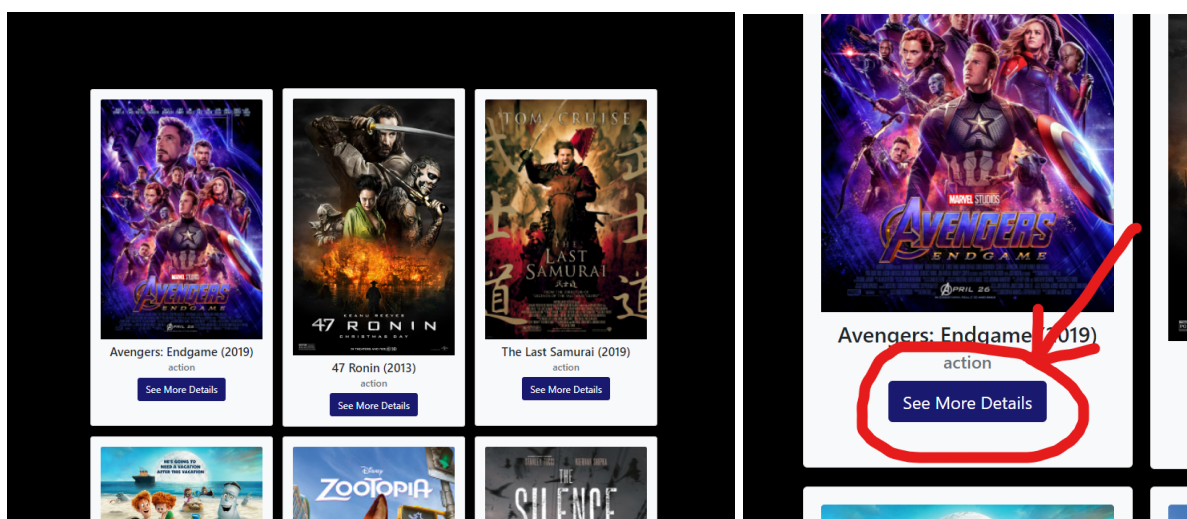
Once a user has signed in, users can choose whether they would like to search with criteria (name, genre and released date) or search with no criteria.



If the user searches for movies with no criteria, it will shows all movie in the website.



If the user searches for movies with a criteria, it will show only movies that match the selected criteria. This is an example of the results of searching movies by released year “2018”.

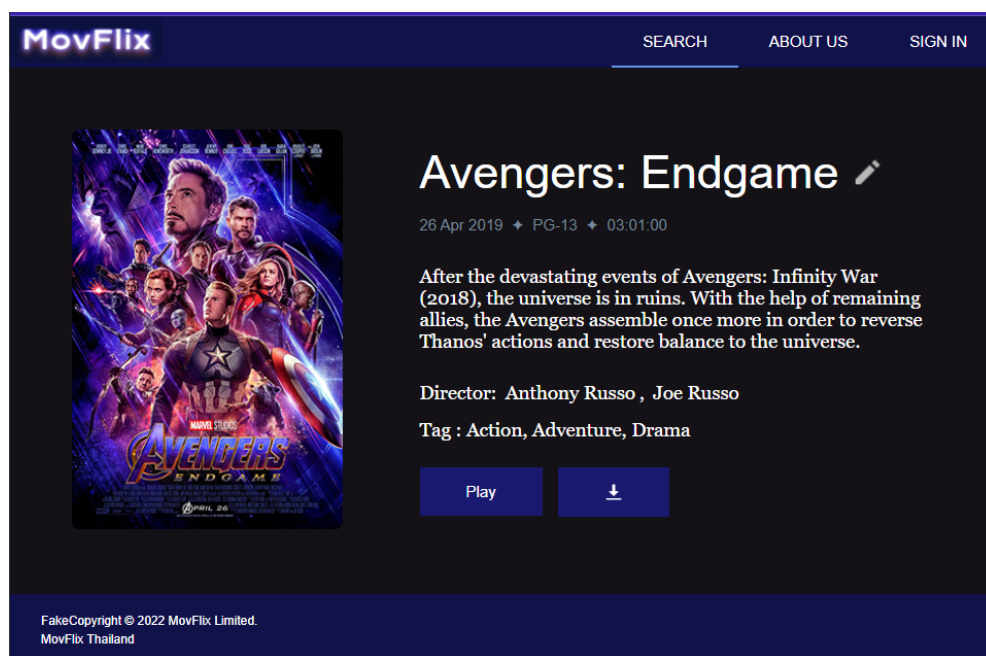


When the user clicks the “See More Details” button, it will go to the movie detail page of each movie.

5. Movie Details Page

Movie details page shows the movie details of each movie after the user clicks on the “See More Details” button in the search movie page. Movie details consist of the movie's name, released date, rate, runtime, description, director's name and genre. Below the movie details, there are play button and download button.

Phase I



Phase II



Name: Avengers: Endgame

Releases date: Fri Apr 26 2019 00:00:00 GMT+0700 (เวลาอินโดจีน)

Rate: PG-13

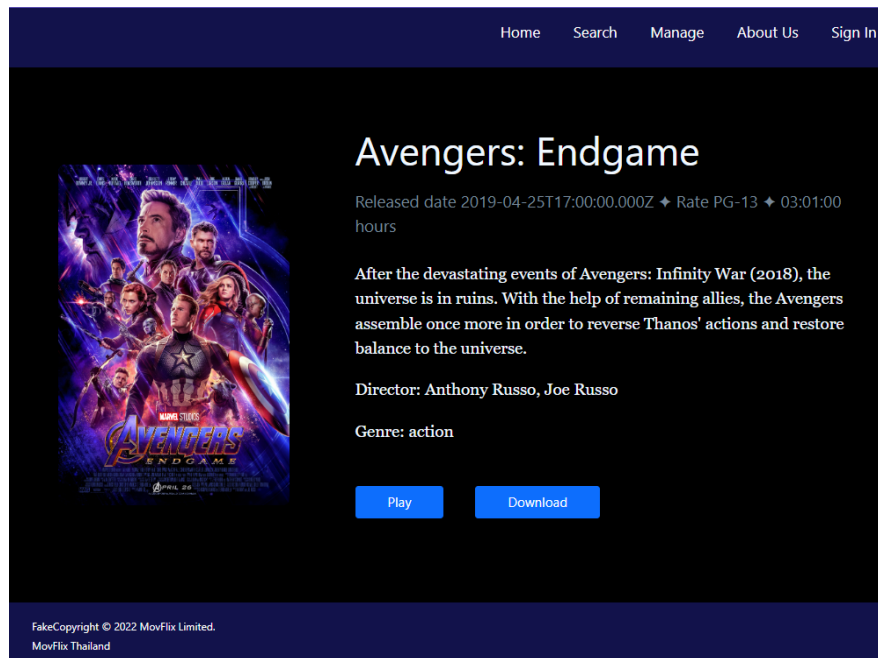
Runtime: 03:01:00

Description: After the devastating events of Avengers: Infinity War (2018), the universe is in ruins. With the help of remaining allies, the Avengers assemble once more in order to reverse Thanos' actions and restore balance to the universe.

Director: Anthony Russo, Joe Russo

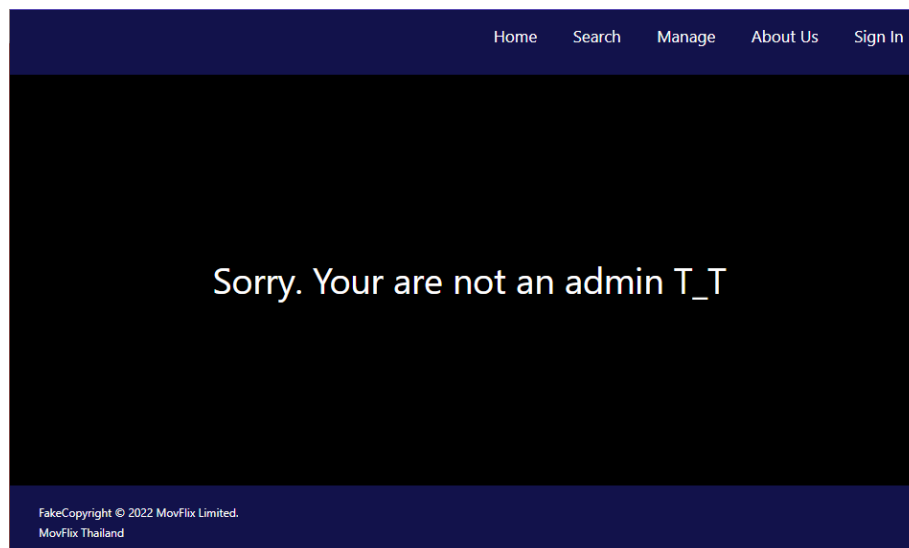
Tag: action

Phase III: <http://localhost:3000/moviedetail/:id>



6. Manage Page (for normal user)

Phase III: <http://localhost:3000/manage>



Manage page is for the administrator to manage movies and users of the website such as select, insert, update, and delete. The user must sign in first before accessing this page. If the user has not signed in, it will redirect to the sign in page automatically. But if the user who accesses this page is a normal user, they will see the message notifying them that they are not an administrator.

Administrator Pages

1. Sign In Page (for administrator)

Phase I & Phase II

MovFlix SEARCH ABOUT US SIGN IN

Sign in

Email

Password

[Forgot password?](#)

[Sign In](#)

Or Sign Up using
[SIGN UP](#)

FakeCopyright © 2022 MovFlix Limited.
MovFlix Thailand

Phase III: <http://localhost:3000/signin>

Home Search Manage About Us Sign In

Sign In

Email

Password

[Sign In](#)

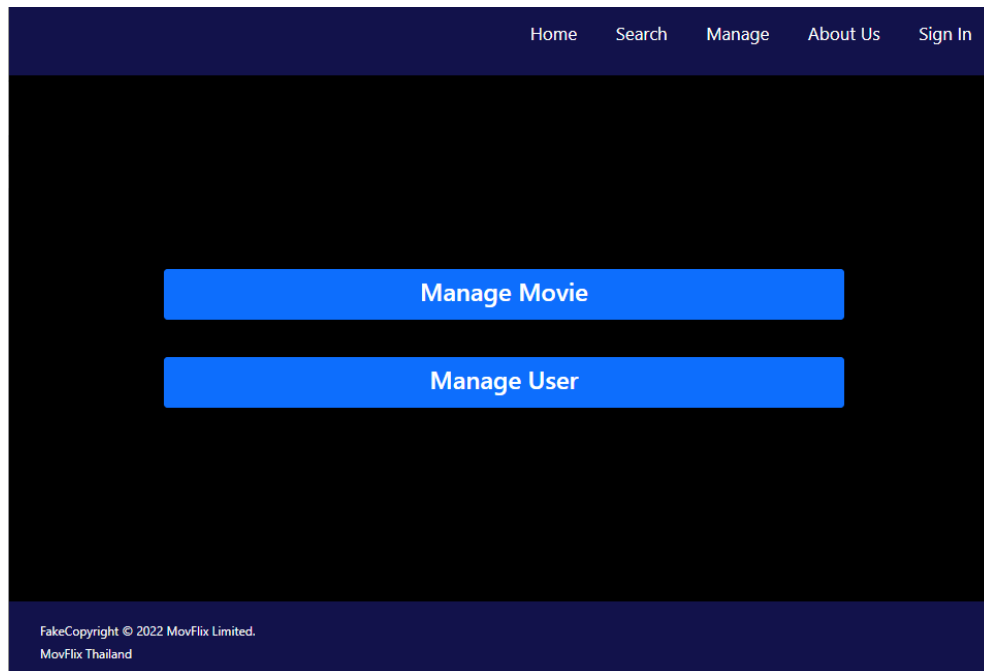
[Sign out](#)

FakeCopyright © 2022 MovFlix Limited.
MovFlix Thailand

Sign in page for the administrator works the same as the sign in page for normal users.

2. Manage Page (for administrator)

Phase III: <http://localhost:3000/manage>



Manage page is for the administrator to manage movies and users of the website such as select, insert, update, and delete. The user must sign in first before accessing this page. If the user has not signed in, it will redirect to the sign in page automatically. And if the user who accesses this page is an administrator, there will be two buttons: “Manage Movie” and “Manage User”. Manage Movie button links to the manage movie pages and Manage User button links to the manage user page.

3. Manage Movie Page

Phase III: <http://localhost:3000/manage/movie>

Home

Search

Manage

About Us

Sign In

Manage Movie

Search Movie:

Search...

Search movie by...

Search

Movie ID:

Type movie ID here...

Movie name:

Type movie name here...

Image source:

Type movie image here...

Released date:

mm/dd/yyyy

Run time:

Type run time in the format hh:mm:ss

Director:

Type movie director here...

Rate:

Type movie rate here...

Genre:

Type movie tag here...

Description:

Type movie description here...

Insert

Update

Delete

Search results

All movies

Id: 130001

Avengers: Endgame

Image source: https://m.media-amazon.com/images/M/MV5BMjc0MDU2ODcwNjU5MTk1ZW5kZW50aWZlbnVUSAM6MDE@_V1_1.jpg

Released date: 2019-04-25

Run time: 03:01:00

Director: Anthony Russo, Joe Russo

Movie rate: PG-13

Genre: action

Description: After the devastating events of Avengers: Infinity War (2018), the universe is in ruins. With the help of remaining allies, the Avengers assemble once more in order to reverse Thanos' actions and restore balance to the universe.

Id: 130002

47 Ronin

Image source: https://m.media-amazon.com/images/M/MV5BMjU2MjU0ODcwLTk1MjM1pgLXk1000_.jpg

Released date: 2013-12-05

Run time: 02:08:00

Director: Carl Lougher

Movie rate: PG-13

Genre: action

Description: A band of samurai set out to avenge the death and dishonor of their master at the hands of a ruthless shogun.

Id: 130003

The Last Samurai

Image source: https://m.media-amazon.com/images/M/MV5BMjY2MDE3MDE0LW50MTk1ZW5kZW50aWZlbnVUSAM6MDE@_V1_1.jpg

Released date: 2010-01-20

Run time: 02:34:00

Director: Edward Zwick

Movie rate: R

Genre: action

Description: An American military advisor embarks on the Samurai culture he was hired to destroy after he is captured in battle.

Id: 130004

Hotel Transylvania 3: Summer Vacation

Image source: https://m.media-amazon.com/images/M/MV5BMjY2MDE3MDE0LW50MTk1ZW5kZW50aWZlbnVUSAM6MDE@_V1_1.jpg

Released date: 2018-07-12

Run time: 01:31:00

Director: Genndy Tartakovsky

Movie rate: PG

Genre: comedy

Description: Count Dracula and company participate in a cruise for sea-loving monsters, unaware that their boat is being commanded by the monster-hating Van Helsing family.

Id: 130005

Zootopia

Image source: https://lumiere-a.akamaihd.net/v1/images/movie_poster_zootop-4-306a762.jpg

Released date: 2016-04-20

Run time: 01:48:00

Director: Rich Moore, Byron Howard

Movie rate: PG

Genre: comedy

Description: In a city of anthropomorphic animals, a rookie bunny cop and a cynical con artist fox must work together to uncover a conspiracy.

Id: 130006

The silence

Image source: https://m.media-amazon.com/images/M/MV5BMjY2MDE3MDE0LW50MTk1ZW5kZW50aWZlbnVUSAM6MDE@_V1_1.jpg

Released date: 2019-05-15

Run time: 01:30:00

Director: John R. Leonetti

Movie rate: PG-16

Genre: horror

Id: 130007

A WHISKER AWAY

Image source: https://m.media-amazon.com/images/M/MV5BMjY2MDE3MDE0LW50MTk1ZW5kZW50aWZlbnVUSAM6MDE@_V1_1.jpg

Released date: 2020-06-17

Run time: 01:45:00

Director: Linneth Sato, Tomotake Shibayama

Movie rate: PG

Genre: fantasy

Id: 130008

Godzilla: King of the Monsters

Image source: https://m.media-amazon.com/images/M/MV5BMjY2MDE3MDE0LW50MTk1ZW5kZW50aWZlbnVUSAM6MDE@_V1_1.jpg

Released date: 2019-05-30

Run time: 02:11:00

Director: Michael Dougherty

Movie rate: PG-13

Id: 130009

Ghost Wife

Image source: https://m.media-amazon.com/images/M/MV5BMjY2MDE3MDE0LW50MTk1ZW5kZW50aWZlbnVUSAM6MDE@_V1_1.jpg

Released date: 2018-11-21

Run time: 01:40:00

Director: Mark Yimmanabon

Movie rate: PG-16

Genre: horror

Id: 130010

Pee Mak

Image source: https://m.media-amazon.com/images/M/MV5BMjY2MDE3MDE0LW50MTk1ZW5kZW50aWZlbnVUSAM6MDE@_V1_1.jpg

Released date: 2013-03-27

Run time: 01:55:00

Director: Benjaporn Phianthanakun

Movie rate: PG-16

Manage movie page is for administrators to manage all movies of the website. There are three parts in this page which are a form to enter some information, the part showing all search results, and the part showing all movies that are provided in the web application.

Manage Movie

Search Movie:

Search movie by... ▾

Search movie by...

No criteria

Movie ID

Movie name

Genre

Movie ID:

Movie name:

Image source:

Search

The image displays a horizontal row of six movie posters. The first poster is for 'Avengers: Endgame', showing the main cast in their superhero suits. The second poster is for 'The Last Samurai', featuring Tom Cruise in a samurai outfit. The third poster is for 'Teenage Mutant Ninja Turtles', showing the four turtles in their iconic colors. The fourth poster is for 'The Last Samurai', showing Tom Cruise in a samurai outfit. The fifth poster is for 'Teenage Mutant Ninja Turtles', showing the four turtles in their iconic colors. The sixth poster is for 'The Last Samurai', showing Tom Cruise in a samurai outfit.

Example of results of searching by genre “action” in the search result part.

Manage Movie

Search Movie:

Search movie by...

Search

Movie ID:

Movie name:

Image source:

Released date:

Run time:

Director:

Rate:

Genre:

Description:

Insert

Update

Delete

Moreover, the administrator can insert or update the movie by entering the information into the input boxes and clicking the insert or update button. The administrator can delete the movie by entering the movie ID and clicking the delete button. All changes will automatically affect the all movies part without refreshing the page.

4. User Manager Page

Phase III: <http://localhost:3000/manage/user>

[Home](#)
[Search](#)
[Manage](#)
[About Us](#)
[Sign In](#)

Manage User

Search User:

Search user by...

User ID:

First name:

Last name:

Email:

Password:

Role:

Search results

All users

User ID	Firstname	Lastname	Email	Encoded Password	Role
100001	Karl	Schneider	karl.k@gmail.com	\$2b\$10\$epfk.vtAHVvTGt1odeK06SDMuIQvwQ8nps5Qh1aG4d1TgPou87e	normal user
100002	Leadele	Jones	lea_jl@gmail.com	\$2b\$10\$5Hq.Wl6v.T1t8mWG2VQ4Q7y18WH4CJME3vu5Vyany8H8m8p8Lom	normal user
100003	Hiroyuki	Sawano	sh-pro@gmail.com	\$2b\$10\$Ury6C.hrokz6rhj2nA.343Qw3h30Ufem2LXph5J5tpkUGPaw	normal user
100004	Thao	Magath	Marley.fornwer@gmail.com	\$2b\$10\$4L2aww3FyYvP4HLZWHwQ5/EV0D9vxtH3oMZVUW1WUUM7KcUm	normal user
100005	Mary	Somers	Horizon.dwan@gmail.com	\$2b\$10\$5/0kzmM74YQdUCp1868wh57ed85fYU6DvqVCYUPL1v097Cc0KG	normal user
100006	Walter	Fitzroy	Fuse.Hellteah@gmail.com	\$2b\$10\$2v9WVv2a0A6WHh7f4Y2z5QD6yflBQjPW24EVkva7Uhc.TnC6	normal user
100007	Kari	Imahara	Wlk_wings@gmail.com	\$2b\$10\$VkuJfPezgo7Z7BQyRQ28B6AdHapryKuycuQ5T5edRvVvW6	normal user
100008	Obi	Estelaim	Seer.Hearbeat@gmail.com	\$2b\$10\$6xowR6dNf5ndyU50WecH9b3qf8HwQJQK210806wzbt6n7C	normal user
100009	Ramya	Pawikh	Rampant.matin@gmail.com	\$2b\$10\$9M5pplwK9APWz5Q5.zFv35rWVCx0TgGQm7zfualZ5DNB	normal user
100010	Alexander	Masonwell	Cautic_trap@gmail.com	\$2b\$10\$UfMyQhWwxyPawY7N6eulDVkBAhcTg5m0Dc7E0ZD5c51ogVKG	normal user
100011	Kaleb	Cross	Revenant_silence@gmail.com	\$2b\$10\$z2M8q.N9yyp2hyqJzpfC0BfzHqWx3QfNwmdqWVsd2toN6b67a	normal user
100001	Pongprop	Lapikai	pongprop.lap@student.mahidol.ac.th	\$2b\$10\$.v8M1bw6A27.vtGw1turs/hK2.vkag8KvK3Eh9QJf5N35j6	admin
100002	Kittapon	Lerdpayub	kittapon.ler@student.mahidol.ac.th	\$2b\$10\$hw5QNU0tmX91L5A3dV17v03vMgTY8Ck/YfBwA.d0QjU3qm	admin
100003	Paveena	Kumrerdpun	paveena@student.mahidol.ac.th	\$2b\$10\$xtYJN6K7U718hTACU55r080AWN4vnbC35qbjyNfLjh1m8B4hTy	admin
100004	Karin	Prakitsavit	karin.pra@student.mahidol.ac.th	\$2b\$10\$HprQ3baAPy6b7f8d9T0dY3e8UKH0Q2K_Rn1rpfH5xQzJM8dxt	admin

FakeCopyright © 2022 Moofix Limited.
 Moofix Thailand

Manage user page is for administrators to manage all users of the web application. There are three parts in this page which are a form to enter some information, the part showing all search results, and the part showing all users that are the authentic user of the web application.

Manage User

Search User:

Search...

Search user by... ▼ Search

User ID:

Type user ID here

Manage User

Search User:

Search...

Search user by... ▼ Search

- Search user by...
- No criteria
- User ID
- Firstname
- Role (Type 'admin' or 'normal user')

Type first name here....

In this form, the administrator can search the user by a criteria (user ID, user's first name and role), which shows only users that match the selected criteria, or search without criteria, which shows all authentic users in the web application. The search results will be shown in the search results part.

Search results

ID900001 Pongpop Lapvikai Email: pongpop.lap@student.mahidol.ac.th Encoded password: \$2b\$10\$.r8r1MTbsWbAZFxrGwzTiur o/nK2.vLxglKvuKbEKt9O/FSIN3uj6 Role: admin	ID900002 Kittapon Lerdpayub Email: kittapon.ler@student.mahidol.ac.th Encoded password: \$2b\$10\$hs/SQNU0tmX9TIL5ASUleV 17v03xMg1Yf8Ok/YfBwA.diOgU3qz m Role: admin	ID900003 Paveena Kumnerdpun Email: paveena@student.mahidol.ac.th Encoded password: \$2b\$10\$xtY.RNoK7U/1thTACU55rO 80AWN4VnbC9SfqbjmNF8Jh.Im6B4 h1y Role: admin	ID900004 Kanin Prakaikowit Email: kanin.pra@student.mahidol.ac.th Encoded password: \$2b\$10\$hjqpQlrzbAP/v/b7TdP8TOd Y3e9UKHtQQE./Li7mHYzzQuM9xE zK Role: admin
---	--	--	--

Example of results of searching by role "admin" in the search result part.

Manage User

Search User:

Search user by... ▼ Search

User ID:

First name:

Last name:

Email:

Password:

Role:

Insert Update Delete

Moreover, the administrator can insert or update the user by entering the information into the input boxes and clicking the insert or update button. The administrator can delete the user by entering the user ID and clicking the delete button. All changes will automatically affect the all users part without refreshing the page.

All users					
User ID	Firstname	Lastname	Email	Encoded Password	Role
100001	Karl	Schneider	karls@gmail.com	\$2b\$10\$epIk..vrahVu1Gc1odeX065DMu0/wQ6cnp5QHTaG4dTfgPou87e	normal user
100002	Leadale	Jones	lea_j@gmail.com	\$2b\$10\$5hQ.WieBv.T1sRmWGDVd407y18WHACjMEIw5Vjyag6Hrm8pBLom	normal user
100003	Hiroyuki	Sawano	sh.pro@gmail.com	\$2b\$10\$UrgyECrkoxZxhg2EaA.145Qxw7a30Uhmz2LxPHJ5Rp4uGPAw.	normal user
100004	Theo	Magath	Marley.forever@gmail.com	\$2b\$10\$1Zsaw3Vjy4P4HLZWHeO5iEV06FvthXBoM2VvWtWU06M7KUm	normal user
100005	Mary	Somers	Horizon_dear@gmail.com	\$2b\$10\$5/0XzmM14YODUcpT868jeh57xoB5YUbdVqVCYUPLTvo97OcOKG	normal user
100006	Walter	Fitzroy	Fuse_HellYeah@gmail.com	\$2b\$10\$2v9VVvC2aD6WHh7F4Y2rSQ0fyRbQIPW24EArsv7IZnC1Hx6	normal user
100007	Kairi	Imahara	Valk_wings@gmail.com	\$2b\$10\$Vizu7uFRezgW0729eDyRtoQ88z6AdHaplyXuycuQ51SdrFvVcWL6	normal user
100008	Obi	Edolasim	Seer_Heartbeat@gmail.com	\$2b\$10\$16rceR6GzNF5n0Y/u59vRecHPb3jgIRRwQJ/OR20808fv2bRnE7C	normal user
100009	Ramya	Parekh	Rampart_mates@gmail.com	\$2b\$10\$K9Mij5rphLwX9APWz5Q5.zVv35rWICxDigGGOn.7zFualZSZN6	normal user
100010	Alexander	Maxwell	Caustic_trap@gmail.com	\$2b\$10\$UfMyQsIKWwyP6wY7NaEuDV8BAtCf95mJdC7EOZDS6t1ogVKG	normal user
100011	Kaleb	Cross	Revenant_silence@gmail.com	\$2b\$10\$12MRqC1N9yypZhyqZjpfC0bfEzHQWXR8D/NWmdiqwVkd2toNNSbE1a	normal user
900001	Pongpop	Lapvikai	pongpop.lap@student.mahidol.ac.th	\$2b\$10\$5/8rIMTbsWbAZFrGwzTiuro/nK2.vLug/KiuvKbEk90/FSIN3uj6	admin
900002	Kittapon	Lerdprayub	kittapon.ler@student.mahidol.ac.th	\$2b\$10\$hs/SQN0U0mx9TIL545t0v17v03MgTYH0k/YBWA.d/CgU3qzm	admin
900003	Paveena	Kumnerdun	paveena@student.mahidol.ac.th	\$2b\$10\$XtY.RNoK7U/1hTACU55iO804WN4VncC95fqbJmNF8h.Im684h1y	admin
900004	Kanin	Prekaikovit	kanin.pre@student.mahidol.ac.th	\$2b\$10\$hhjQrbsAP/vb7t6P8T0dr3eBUKtQGE./Lr7mHYzzQuN9xEzK	admin

FakeCopyright © 2022 MovFix Limited.
MovFix Thailand

This is all users part.

Web Services

Authentication Service

```

app.post('/signin/auth', function(req, res){
  dbConn.execute(
    "SELECT * FROM user WHERE Email = ?", [req.body.email],
    function(err, user, field) {
      if (err) throw error;
      if (user.length == 0) {
        return res.status(400).json({error: true, message: 'Sign In Failed.'});
      }
      // compare password with decoded password in database
      bcrypt.compare(req.body.password, user[0].User_Password, function(err, isSignIn){
        if (err) return res.status(400).send({ error: true, message: 'error'});
        if(isSignIn){ // if authentic
          // generate token for checking authorization
          let token = jwt.sign(
            {email : user[0].Email, role : user[0].User_Role},
            process.env.Secret,
            {expiresIn: '300'}
          );
          res.status(200).json({ error: false, message: 'Sign in success.', accessToken: token, role: user[0].User_Role});
        }
        else{
          return res.status(400).json({ error: true, message: 'Sign In Failed. Email or Password may be incorrect'});
        }
      });
    }
  );
});

```

This service is for checking whether the user that signed in to the MovFlix website is authentic or not. Once the service receives the request that contains email and password from the sign in page. It will first check if the email is in the database. If the database has this email, then it checks the password with the encoded password in the database using bcrypt package. If it matches, it will generate a token and return the token together with the response.

Search Movie Service

1. Search all movies (No criteria)

```

app.get('/search/movies/all/',function (req, res){ //No criteria (return all results)
  dbConn.query('SELECT * FROM movie', function (error, results) {
    if(error) throw error;
    return res.send({error: false, data: results, message: 'Movie retrieved.'});
  });
});

```

This service is for selecting all movies in the database using method GET. Once it receives the request from the client side, it will select all movies and then return a response that contains error, data, and message in JSON object format.

2. Search movie by name

```
app.get('/search/movies/name/:name', function(req, res){
  let movie_name = req.params.name;
  if(!movie_name){
    return res.status(400).send({ error: true, message: 'Please provide movie name'});
  }
  movie_name = '%' + movie_name + '%'; // to be able to search only some part of movie name
  dbConn.query('SELECT * FROM movie WHERE Movie_Name LIKE ?', movie_name, function (error, results) {
    if(error) throw error;
    return res.send({error: false, data: results, message: 'Movie retrieved.'});
  });
});
```

This service is for selecting targeted movies via movie's name in the database using method GET. Once it receives the request, it will check whether the movie's name is received or not. If not it will response that there is an error and the message to ask the user to input the movie's name. After that it will modify the movie's name string by adding the '%' to make the database query be able to search this movie name string from the full movie's name in the database so that users do not need to enter the full name of each movie. Then it will select all the movies with provided name and then return response that contains error, data, and message in JSON object format.

3. Search movie by genre

```
app.get('/search/movies/tag/:tag', function(req, res){
  let movie_tag = req.params.tag;
  if(!movie_tag){
    return res.status(400).send({ error: true, message: 'Please provide movie tag'});
  }
  movie_tag = '%' + movie_tag + '%';
  dbConn.query('SELECT * FROM movie WHERE Tag LIKE ?', movie_tag, function (error, results) {
    if(error) throw error;
    return res.send({error: false, data: results, message: 'Movie retrieved.'});
  });
});
```

This service is for selecting targeted movies via movie's tag in the database using method GET. Once it receives the request, it will check whether the movie's tag is received or not. If not it will response that there is an error and the message to ask the user to input the movie's tag. After that it will modify the movie's tag string by adding the '%' on both side on original string to make the database query be able to search this movie's tag string from the full movie's name in database so that user do not need to enter the full tag of the movie. Then it will select all the movies with provided tag and then return response that contains error, data, and message in JSON object format.

4. Search movie by released year

```
app.get('/search/movies/year/:year', function(req, res){
  let movie_year = req.params.year;
  if(!movie_year){
    return res.status(400).send({ error: true, message: 'Please provide released year of movie'});
  }
  dbConn.query('SELECT * FROM movie WHERE YEAR(Release_Date) = ?', movie_year, function (error, results) {
    if(error) throw error;
    return res.send({error: false, data: results, message: 'Movie retrieved.'});
  });
});
```

This service is for selecting targeted movies via movie's released date in the database using method GET. Once it receives the request, it will check whether the movie's released date is received or not. If not it will respond that there is an error and the message to ask the user to input the movie's release date. Then it will select all the movies with the provided date and then return a response that contains error, data, and message in JSON object format.

User Manager Service

1. Add new user

```
app.post('/user/add', function (req, res){
  let user = req.body.user;
  if(!user){
    return res.status(400).send({ error: true, message: 'Please provide user information'});
  }
  // encoded password and store information into database
  bcrypt.hash(user.User_Password, saltRounds, function(err, hash){
    dbConn.execute("INSERT INTO user (User_ID, First_Name, Last_Name, Email, User_Password, User_Role) VALUES (?, ?, ?, ?, ?, ?)",
    [user.User_ID, user.First_Name, user.Last_Name, user.Email, hash, user.User_Role],
    function (error, results) {
      if(error){ res.status(400).send({ error: true, message: 'error'});}
      return res.send({error: false, message: 'New user has been created successfully.'});
    });
  });
});
```

This service is for adding an user to the database using method POST. Once it receives the request, it will check whether the user information is fully received or not. If not it will response that there is an error and the message to ask the user to fill the information again. After that it will encode the received password. Then it will add or insert this new user into the database and then return a response that contains error, data, and message in JSON object format.

2. Update user information

```
app.put('/user/update', function (req, res){
  let user_id = req.body.user.User_ID;
  let user = req.body.user;

  if(!user_id || !user){
    return res.status(400).send({ error: user, message: 'Please provide user information' });
  }
  bcrypt.hash(user.User_Password, saltRounds, function(err, hash){
    user.User_Password = hash;
    dbConn.query('UPDATE user SET ? WHERE User_ID = ?', [user, user_id], function (error, results) {
      if(error) throw error;
      return res.send({error: false, data: results.affectedRows, message: 'This user has been updated successfully.'});
    });
  });
});
```

This service is for updating user's information to a targeted user via user id in the database using method PUT. Once it receives the request, it will check whether the user ID and other information is received or not. If not it will respond that there is an error and the message to ask the user to fill the user's information again. After that it will encode the received password. Then it will update targeted user information in the database and then return a response that contains error, data, and message in JSON object format.

3. Delete a user

```
app.delete('/user/delete', function(req, res){
  let user_id = req.body.User_ID;

  if(!user_id){
    return res.status(400).send({ error: true, message: 'Please provide User_ID' });
  }
  dbConn.query('DELETE FROM user WHERE User_ID = ?', [user_id], function (error, results) {
    if(error) throw error;
    return res.send({error: false, data: results.affectedRows, message: 'This user has been deleted successfully.'});
  });
});
```

This service is for deleting a targeted user via user id in the database using method PUT. Once it receives the request, it will check whether the user id is received or not. If not it will respond that there is an error and the message to ask the user to fill the user id again. Then it will delete that targeted user in the database and then return a response that contains error, data, and message in JSON object format.

4. Select a user by user ID

```
app.get('/user/select/id/:id', function(req, res){
  let user_id = req.params.id;
  if(!user_id){
    return res.status(400).send({ error: true, message: 'Please provide User_ID'});
  }
  dbConn.query('SELECT * FROM user WHERE User_ID = ?', user_id, function (error, results) {
    if(error) throw error;
    return res.send({error: false, data: results, message: 'User retrieved.'});
  });
});
```

This service is for selecting users via user id in the database using method GET. Once it receives the request, it will check whether the user id is received or not. If not it will respond that there is an error and the message to ask the user to input the user id again. Then it will select the user with the provided id and then return a response that contains error, data, and message in JSON object format.

5. Select a user by first name

```
app.get('/user/select/firstname/:firstname', function(req, res){
  let user_firstname = req.params.firstname;
  if(!user_firstname){
    return res.status(400).send({ error: true, message: 'Please provide First_Name'});
  }
  user_firstname = '%' + user_firstname + '%';
  dbConn.query('SELECT * FROM user WHERE First_Name LIKE ?', user_firstname, function (error, results) {
    if(error) throw error;
    return res.send({error: false, data: results, message: 'User retrieved.'});
  });
});
```

This service is for selecting users via user's first name in the database using method GET. Once it receives the request, it will check whether the user's first name is received or not. If not it will respond that there is an error and the message to ask the user to input the user's first name again. After that it will modify the user's first name string by adding the '%' on both sides of the original string to make the database query be able to search this user's first name string from the full user's first name in the database so that users do not need to enter the full first name. Then it will select the user with the provided first name and then return a response that contains error, data, and message in JSON object format.

6. Select a user by role

```
app.get('/user/select/role/:role', function(req, res){
  let user_role = req.params.role;
  if(!user_role){
    return res.status(400).send({ error: true, message: 'Please provide User_Role' });
  }
  dbConn.query('SELECT * FROM user WHERE User_Role = ?', user_role, function (error, results) {
    if(error) throw error;
    return res.send({error: false, data: results, message: 'User retrieved.'});
  });
});
```

This service is for selecting users via user role in the database using method GET. Once it receives the request, it will check whether the user role is received or not. If not it will respond that there is an error and the message to ask the user to input the user role again. Then it will select all users with the provided role and then return a response that contains error, data, and message in JSON object format.

7. Select all users

```
app.get('/user/selectall', function (req, res){
  dbConn.query('SELECT * FROM user', function (error, results) {
    if(error) throw error;
    return res.send({error: false, data: results, message: 'User list.'});
  });
});
```

This service is for selecting all the users in the database using method GET. Once it receives the request. It will select all users and then return a response that contains error, data, and message in JSON object format.

Movie Manager Service

1. Add new movie

```
app.post('/movie/add', function (req, res){
  let movie = req.body.movie;

  if(!movie){
    return res.status(400).send({ error: true, message: 'Please provide movie information' });
  }
  dbConn.query("INSERT INTO movie SET ?", movie, function (error, results) {
    if(error) throw error;
    return res.send({error: false, data: results.affectedRows, message: 'New movie has been created successfully.'});
  });
});
```

This service is for adding a movie to the database using method POST. Once it receives the request, it will check whether the movie's information is fully received or not. If not it will response that there is an error and the message to ask the user to fill the

information again. Then it will add or insert this new user into the database and then return a response that contains error, data, and message in JSON object format.

2. Update movie information

```
app.put('/movie/update', function (req, res){
  let movie_id = req.body.movie.Movie_ID;
  let movie = req.body.movie;

  if(!movie_id || !movie){
    return res.status(400).send({ error: movie, message: 'Please provide movie information' });
  }
  dbConn.query('UPDATE movie SET ? WHERE Movie_ID = ?', [movie, movie_id], function (error, results) {
    if(error) throw error;
    return res.send({error: false, data: results.affectedRows, message: 'This movie has been updated successfully.'});
  });
});
```

This service is for updating movie's information to a targeted movie via movie id in the database using method PUT. Once it receives the request, it will check whether the movie id and other information is received or not. If not it will respond that there is an error and the message to ask the user to fill the movie's information again. Then it will update the targeted movie's information in the database and then return a response that contains error, data, and message in JSON object format.

3. Delete a movie

```
app.delete('/movie/delete', function(req, res){
  let movie_id = req.body.Movie_ID;

  if(!movie_id){
    return res.status(400).send({ error: true, message: 'Please provide Movie_ID' });
  }
  dbConn.query('DELETE FROM movie WHERE Movie_ID = ?', [movie_id], function (error, results) {
    if(error) throw error;
    return res.send({error: false, data: results.affectedRows, message: 'This movie has been deleted successfully.'});
  });
});
```

This service is for deleting a targeted movie via movie id in the database using method PUT. Once it receives the request, it will check whether the user id is received or not. If not it will respond that there is an error and the message to ask the user to fill the movie id again. Then it will delete that targeted movie in the database and then return a response that contains error, data, and message in JSON object format.

4. Select a movie by movie ID

```
app.get('/movie/select/id/:id',function(req, res){
  let movie_id = req.params.id;

  if(!movie_id){
    return res.status(400).send({ error: true, message: 'Please provide Movie_ID'});
  }
  dbConn.query('SELECT * FROM movie WHERE Movie_ID = ?', movie_id, function (error, results) {
    if(error) throw error;
    return res.send({error: false, data: results, message: 'Movie retrieved.'});
  });
});
```

This service is for selecting movies via movie id in the database using method GET. Once it receives the request, it will check whether the movie id is received or not. If not it will respond that there is an error and the message to ask the user to input the movie id again. Then it will select the movie with the provided id and then return a response that contains error, data, and message in JSON object format.

5. Select a movie by movie name

```
app.get('/movie/select/name/:name',function(req, res){
  let movie_name = req.params.name;
  if(!movie_name){
    return res.status(400).send({ error: true, message: 'Please provide movie name'});
  }
  movie_name = '%' + movie_name + '%'; // to be able to serch only some part of movie name
  dbConn.query('SELECT * FROM movie WHERE Movie_Name LIKE ?', movie_name, function (error, results) {
    if(error) throw error;
    return res.send({error: false, data: results, message: 'Movie retrieved.'});
  });
});
```

This service is for selecting a movie via movie's name in the database using method GET. Once it receives the request, it will check whether the movie's name is received or not. If not it will respond that there is an error and the message to ask the user to input the movie's name again. After that it will modify the movie's name string by adding the '%' on both sides of the original string to make the database query be able to search this movie's name string from the full movie's name in the database so that users do not need to enter the full movie's name. Then it will select the movie with the provided name and then return a response that contains error, data, and message in JSON object format.

6. Select a movie by movie genre

```
app.get('/movie/select/tag/:tag',function(req, res){
  let movie_tag = req.params.tag;
  if(!movie_tag){
    return res.status(400).send({ error: true, message: 'Please provide movie tag'});
  }
  movie_tag = '%' + movie_tag + '%';
  dbConn.query('SELECT * FROM movie WHERE Tag LIKE ?', movie_tag, function (error, results) {
    if(error) throw error;
    return res.send({error: false, data: results, message: 'Movie retrieved.'});
  });
});
```

This service is for selecting a movie via movie's tag or genre in the database using method GET. Once it receives the request, it will check whether the movie's tag is received or not. If not it will respond that there is an error and the message to ask the user to input the movie's tag again. After that it will modify the movie's tag string by adding the '%' on both sides of the original string to make the database query be able to search this movie's tag string from the full movie's name in the database so that users do not need to enter the full movie's tag. Then it will select the movie with the provided tag and then return a response that contains error, data, and message in JSON object format.

7. Select all movies

```
app.get('/movie/selectall', function (req, res){
  dbConn.query('SELECT * FROM movie', function (error, results) {
    if(error) throw error;
    return res.send({error: false, data: results, message: 'Movie list.'});
  });
});
```

This service is for selecting all the movies in the database using method GET. Once it receives the request. It will select all movies and then return a response that contains error, data, and message in JSON object format.

Web Services Test Results

Authentication Service

The screenshot shows the Postman interface for a REST client. The top bar displays the URL `http://localhost:3030/signin/auth`. Below it, the method is set to `POST`. The body tab is selected, showing a JSON payload:

```
{  "email": "paveena@student.mahidol.ac.th",  "password": "mko@98"}
```

The bottom section shows the response status as `200 OK`, with a time of `62 ms` and size of `546 B`. The response body is displayed in the Pretty view:

```
{  "error": false,  "message": "Sign in success.",  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJlbWFnbnBpbCInBhdmVlbnFAC3RIZGVudC5tYWhpZG9sLmFjLnRoIiwicm9sZSI6ImFkbWwuaWiaWF0eIjoxNjUwNzI1ODkyLCJleHAiOjE2NTA3MjU4OTQ3LjEwMTIK3JnbnB4gBb20gy5YPkWLWRg9-BT8firEimGSw",  "role": "admin"}
```

Search Movie Service

1. Search all movies (no criteria)

2. Search movie by name

GET http://localhost:3030/search/movies/name/2012 Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

Body Cookies Headers (8) Test Results Status: 200 OK Time: 7 ms Size: 686 B Save Response

Pretty Raw Preview Visualize JSON

```

1  {
2    "error": false,
3    "data": [
4      {
5        "Movie_ID": 130011,
6        "Movie_Name": "2012",
7        "Movie_img_src": "2012.jpg",
8        "Release_Date": "2009-11-12T17:00:00.000Z",
9        "Run_Time": "02:38:00",
10       "Director": "Roland Emmerich",
11       "Rate": "PG-13",
12       "Tag": "sci-fi",
13       "Movie_Description": "When a flood of natural disasters begins to destroy the world, a divorced dad desperately tries to save his family
14         by outrunning the cataclysmic chaos."
15     }
16   ],
17   "message": "Movie retrieved."

```

3. Search movie by genre

GET http://localhost:3030/search/movies/tag/action Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

Body Cookies Headers (8) Test Results Status: 200 OK Time: 6 ms Size: 2.2 KB Save Response

Pretty Raw Preview Visualize JSON

```

1  {
2    "error": false,
3    "data": [
4      {
5        "Movie_ID": 130001,
6        "Movie_Name": "Avengers: Endgame",
7        "Movie_img_src": "Avengers_Endgame.jpg",
8        "Release_Date": "2019-04-25T17:00:00.000Z",
9        "Run_Time": "03:01:00",
10       "Director": "Anthony Russo, Joe Russo",
11       "Rate": "PG-13",
12       "Tag": "action",
13       "Movie_Description": "After the devastating events of Avengers: Infinity War (2018), the universe is in ruins. With the help of remaining
14         allies, the Avengers assemble once more in order to reverse Thanos' actions and restore balance to the universe."
15     },
16     {
17       "Movie_ID": 130002,
18       "Movie_Name": "47 Ronin",
19       "Movie_img_src": "47 Ronin.jpg",
20       "Release_Date": "2013-12-05T17:00:00.000Z",
21       "Run_Time": "02:08:00",
22       "Director": "Carl Rinsch",
23       "Rate": "PG-13",
24       "Tag": "action",
25       "Movie_Description": "A band of samurai set out to avenge the death and dishonor of their master at the hands of a ruthless shogun."
26     }

```

4. Search movie by released year

The screenshot shows a REST client interface with a GET request to `http://localhost:3030/search/movies/year/1998`. The response is a JSON object with the following structure:

```
1  {
2    "error": false,
3    "data": [
4      {
5        "Movie_ID": 130014,
6        "Movie_Name": "Deep Impact",
7        "Movie_img_src": "Deep Impact.jpg",
8        "Release_Date": "1998-05-07T17:00:00.000Z",
9        "Run_Time": "02:02:00",
10       "Director": "Mimi Leder",
11       "Rate": "PG-13",
12       "Tag": "sci-fi",
13       "Movie_Description": "With a massive comet threatening to destroy Earth, the U.S. president appoints a steely veteran astronaut to lead a
14         crew into space to destroy it."
15     }
16   ],
17   "message": "Movie retrieved."
18 }
```

The interface includes tabs for Params, Authorization, Headers (6), Body, Pre-request Script, Tests, and Settings. The Body tab is selected, showing the JSON response. The status bar indicates Status: 200 OK, Time: 4 ms, Size: 690 B. A 'Send' button is visible in the top right corner.

User Manager Service

1. Add new user

POST ▼ http://localhost:3030/user/add Send ▼

Params Authorization Headers (8) **Body** Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON ▼ Beautify

```

4 ..... "First_Name": "Peter",
5 ..... "Last_Name": "Parker",
6 ..... "Email": "FriendlyNeighborhood@gmail.com",
7 ..... "User_Password": "spm123",
8 ..... "User_Role": "normal-user"
9 .....

```

Body Cookies Headers (8) Test Results Status: 200 OK Time: 73 ms Size: 334 B Save Response ▼

Pretty Raw Preview Visualize JSON ▼ ⌵

```

1 .....
2 ..... "error": false,
3 ..... "message": "New user has been created successfully."
4 .....

```

	User_ID	First_Name	Last_Name	Email	User_Password	User_Role
▶	100001	Karl	Schneider	karl.s@gmail.com	\$2b\$10\$epIik..vrAhVu1Gc1odeXO65DMuf0/wQ...	normal user
	100002	Leadale	Jones	lea_j@gmail.com	\$2b\$10\$ShQ.WIeBv.T1sRmWG0VD4O7y18WH...	normal user
	100003	Hiroyuki	Sawano	sh.pro@gmail.com	\$2b\$10\$UrQyECrkrokzXnfvj2EnA.14SQwx7s30...	normal user
	100004	Theo	Magath	Marley.forever@gmail.com	\$2b\$10\$tL2sawv3lVyJvP4HLZWHeO5/EVJ06Fxt...	normal user
	100005	Mary	Somers	Horizon_dear@gmail.com	\$2b\$10\$S/lXkzmM14YOdUCpT868jeh57xoB5JY...	normal user
	100006	Walter	Fitzroy	Fuse_HellYeah@gmail.com	\$2b\$10\$Zv9WVVc2a0A6WHh7F4Y2r.SQ0r6yRb...	normal user
	100007	Kairi	Imahara	Valk_wings@gmail.com	\$2b\$10\$Vizu7uFPezgx07Z9bDyRo.QB8z6AdHa...	normal user
	100008	Obi	Edolasim	Seer_Heartbeat@gmail.com	\$2b\$10\$I6rceRs6zNFSn0y/u59xReCHPb3jqIIRR...	normal user
	100009	Ramya	Parekh	Rampart_mates@gmail.com	\$2b\$10\$k9MijSrplvLwX9APWz5QS.zYv35trWIC...	normal user
	100010	Alexander	Maxwell	Caustic_trap@gmail.com	\$2b\$10\$UfMyQsIWKvwyP6xvY7NaEuIDVkBATr...	normal user
	100011	Kaleb	Cross	Revenant_silence@gmail.com	\$2b\$10\$J2MRqCN9yypZhyqIZjsFcObfzHQWX...	normal user
	123456	Peter	Parker	FriendlyNeighborhood@gm...	\$2b\$10\$qLPk2oEbsL9B4dr7pYdCLucQwBzSUBQ...	normal user
	900001	Pongpop	Lapvikai	pongpop.lap@student.mah...	\$2b\$10\$.r8rIMTbsWbAZFxrGwzTiuro/nK2.vLxgl...	admin
	900002	Kittapon	Lerdpayub	kittapon.ler@student.mahi...	\$2b\$10\$hs/SQNU0tmX9TiL5ASTjleV17v03xMg1...	admin
	900003	Paveena	Kumnerdpun	paveena@student.mahidol...	\$2b\$10\$xTY.RNoK7U/1thTACUSrO80AWN4Vn...	admin
	900004	Kanin	Prakaikowit	kanin.pra@student.mahido...	\$2b\$10\$hhjpQIrbzAP/v/b7TdP8TOdY3e9UKHtQ...	admin

2. Update user information

PUT http://localhost:3030/user/update Send

Params Authorization Headers (8) **Body** Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```

1 {
2   "user": {
3     "User_ID": 123456,
4     "First_Name": "Ben",
5     "Last_Name": "Parker",
6     "Email": "deadOldMan@gmail.com",
7     "User_Password": "spm123",
8     "User_Role": "normal user"
9   }
10 }

```

Body Cookies Headers (8) Test Results Status: 200 OK Time: 61 ms Size: 344 B Save Response

Pretty Raw Preview Visualize JSON

```

1 {
2   "error": false,
3   "data": 1,
4   "message": "This user has been updated successfully."
5 }

```

	User_ID	First_Name	Last_Name	Email	User_Password	User_Role
▶	100001	Karl	Schneider	karl.s@gmail.com	\$2b\$10\$epIik..vrAhVu1Gc1odeXO65DMuf0/wQ...	normal user
	100002	Leadale	Jones	lea_j@gmail.com	\$2b\$10\$ShQ.WIeBv.T1sRmWG0VD4O7y18VH...	normal user
	100003	Hiroyuki	Sawano	sh.pro@gmail.com	\$2b\$10\$UrQyECrkrokzXnfvj2EnA.14SQwx7s30...	normal user
	100004	Theo	Magath	Marley.forever@gmail.com	\$2b\$10\$tL2sawv3lVyJvP4HLZWHeO5/EVJ06Fxt...	normal user
	100005	Mary	Somers	Horizon_dear@gmail.com	\$2b\$10\$S/lXkzmM14YOdUCpT868jeh57xoB5JY...	normal user
	100006	Walter	Fitzroy	Fuse_HellYeah@gmail.com	\$2b\$10\$Zv9WVvc2a0A6WHh7F4Y2r.SQ0r6yRb...	normal user
	100007	Kairi	Imahara	Valk_wings@gmail.com	\$2b\$10\$Vizu7uFPezgx07Z9bDyRo.QB8z6AdHa...	normal user
	100008	Obi	Edolasim	Seer_Heartbeat@gmail.com	\$2b\$10\$I6rceRs6zNFSn0y/u59xRecHPb3jqIRR...	normal user
	100009	Ramya	Parekh	Rampart_mates@gmail.com	\$2b\$10\$k9MijSrplvLwX9APWz5QS.zYv35trWIC...	normal user
	100010	Alexander	Maxwell	Caustic_trap@gmail.com	\$2b\$10\$UfMyQsIWKvwyP6xvY7NaEuIDVkBATR...	normal user
	100011	Kaleb	Cross	Revenant_silence@gmail.com	\$2b\$10\$J2MRqCN9yypZhyqIZjsFcObfzHQWX...	normal user
	123456	Ben	Parker	deadOldMan@gmail.com	\$2b\$10\$Iqmm1xWoWn6Qn4KXH15fH.KcFgbIHb...	normal user
	900001	Pongpop	Lapvikai	pongpop.lap@student.mah...	\$2b\$10\$.r8IMTbsWbAZFxrGwzTiuro/nK2.vLxgl...	admin
	900002	Kittapon	Lerdpayub	kittapon.ler@student.mahi...	\$2b\$10\$hs/SQNU0tmX9TiL5ASTjIeV17v03xMg1...	admin
	900003	Paveena	Kumnerdpun	paveena@student.mahidol...	\$2b\$10\$xTY.RNoK7U/1thTACUSSrO80AWN4Vn...	admin
	900004	Kanin	Prakaikowit	kanin.pra@student.mahido...	\$2b\$10\$hhjpQIrbzAP/v/b7TdP8TOdY3e9UKhtQ...	admin

3. Delete a user

DELETE ▼ http://localhost:3030/user/delete Send ▼

Params Authorization Headers (8) **Body** ● Pre-request Script Tests Settings Cookies

● none ● form-data ● x-www-form-urlencoded ● raw ● binary ● GraphQL ● JSON ▼ Beautify

```

1
2
3
4
5

```

Body Cookies Headers (8) Test Results 🌐 Status: 200 OK Time: 13 ms Size: 344 B Save Response ▼

Pretty Raw Preview Visualize JSON ▼ 🔍

```

1
2
3
4
5

```

4. Select a user by user ID

http://localhost:3030/user/select/id/900001 Save ▼ 🔍

GET ▼ http://localhost:3030/user/select/id/900001 Send ▼

Params Authorization Headers (8) **Body** ● Pre-request Script Tests Settings Cookies

Query Params

KEY	VALUE	DESCRIPTION	...	Bulk Edit
Key	Value	Description		

Body Cookies Headers (8) Test Results 🌐 Status: 200 OK Time: 5 ms Size: 529 B Save Response ▼

Pretty Raw Preview Visualize JSON ▼ 🔍

```

1
2
3
4
5
6
7
8
9
10
11
12
13
14

```

5. Select a user by first name

GET ▼ http://localhost:3030/user/select/firstname/Theo Send ▼

Params Authorization Headers (8) Body ● Pre-request Script Tests Settings Cookies

Query Params

KEY	VALUE	DESCRIPTION	...	Bulk Edit
Key	Value	Description		

Body Cookies Headers (8) Test Results ⊞ Status: 200 OK Time: 6 ms Size: 520 B Save Response ▼

Pretty Raw Preview Visualize JSON ▼ ≡

```

1  {
2    "error": false,
3    "data": [
4      {
5        "User_ID": 100004,
6        "First_Name": "Theo",
7        "Last_Name": "Magath",
8        "Email": "Marley.forever@gmail.com",
9        "User_Password": "$2b$10$StL2sawv3lVyJvP4HLZWH05/EVJ06FxtHk9oM2VuWTLwUU6M7KrUm",
10       "User_Role": "normal user"
11     }
12   ],
13   "message": "User retrieved."
14 }

```

6. Select a user by role Admin

http://localhost:3030/user/select/role/admin Save ▼ ✎ 💬

GET ▼ http://localhost:3030/user/select/role/admin Send ▼

Params Authorization Headers (8) Body ● Pre-request Script Tests Settings Cookies

Body Cookies Headers (8) Test Results ⊞ Status: 200 OK Time: 5 ms Size: 1.13 KB Save Response ▼

Pretty Raw Preview Visualize JSON ▼ ≡

```

1  {
2    "error": false,
3    "data": [
4      {
5        "User_ID": 900001,
6        "First_Name": "Pongpop",
7        "Last_Name": "Lapvikai",
8        "Email": "pongpop.lap@student.mahidol.ac.th",
9        "User_Password": "$2b$10$.r8xIMTbsWbAZFxrGwzTiuro/nK2.vLxglKvuKbEKt90/FS1N3uj6",
10       "User_Role": "admin"
11     },
12     {
13       "User_ID": 900002,
14       "First_Name": "Kittapon",
15       "Last_Name": "Lerdpayub",
16       "Email": "kittapon.ler@student.mahidol.ac.th",
17       "User_Password": "$2b$10$hs/SQNU0tmX9TiL6AstJleV17v03xMg1Yf80k/Yf8wA.di0gU3qzm",
18       "User_Role": "admin"
19     },
20     {
21       "User_ID": 900003,
22       "First_Name": "Paveena",
23       "Last_Name": "Kumnerdpun",
24       "Email": "paveena@student.mahidol.ac.th",
25       "User_Password": "$2b$10$XtY.RNoK7U/1thTACUS5r080AwN4VnbC9SfqbjmNF8Jh.Im6B4h1y",
26       "User_Role": "admin"
27     }
28   ]
29 }

```

Normal user

http://localhost:3030/user/select/role/normal user

GET http://localhost:3030/user/select/role/normal user

Params Authorization Headers (8) Body ● Pre-request Script Tests Settings

Body Cookies Headers (8) Test Results

Status: 200 OK Time: 6 ms Size: 2.45 KB Save Response

Pretty Raw Preview Visualize JSON

```

1  {
2    "error": false,
3    "data": [
4      {
5        "User_ID": 100001,
6        "First_Name": "Karl",
7        "Last_Name": "Schneider",
8        "Email": "karl.s@gmail.com",
9        "User_Password": "$2b$10$epIik..vrAhVu1Gc1odeX065DMuf0/wQ6cnp5QHTaG4dTFgPou87e",
10       "User_Role": "normal user"
11      },
12      {
13        "User_ID": 100002,
14        "First_Name": "Leadale",
15        "Last_Name": "Jones",
16        "Email": "lea_j@gmail.com",
17        "User_Password": "$2b$10$ShQ.WIeBv.T1sRmWG0VD407y18WH4CJMEIvu5Vyanj6HRm8p1BLom",
18        "User_Role": "normal user"
19      },
20      {
21        "User_ID": 100003,
22        "First_Name": "Hiroyuki",
23        "Last_Name": "Sawano",
24        "Email": "sh.pro@gmail.com",
25        "User_Password": "$2b$10$UrqyECrkrokzXnfvj2EnA.14SQwx7s30Uhm2LXpHhJ5Rp4uGPAw.",
26        "User_Role": "normal user"
27      }
28    ]
29  }

```

7. Select all users

http://localhost:3030/user/selectall

GET http://localhost:3030/user/selectall

Params Authorization Headers (8) Body ● Pre-request Script Tests Settings

Body Cookies Headers (8) Test Results

Status: 200 OK Time: 13 ms Size: 3.26 KB Save Response

Pretty Raw Preview Visualize JSON

```

82  {
83    "User_Role": "normal user"
84  },
85  {
86    "User_ID": 100011,
87    "First_Name": "Kaleb",
88    "Last_Name": "Cross",
89    "Email": "Revenant_silence@gmail.com",
90    "User_Password": "$2b$10$J2MRqCN9yypZhyqIZjsFc0bfEzHQWx6D/NWmdiqwVkd2toNNSbE1a",
91    "User_Role": "normal user"
92  },
93  {
94    "User_ID": 900001,
95    "First_Name": "Pongpop",
96    "Last_Name": "Lapvikai",
97    "Email": "pongpop.lap@student.mahidol.ac.th",
98    "User_Password": "$2b$10$.r8xIMTbsWbAZFxrGwzTiuro/nK2.vLxg1KvuKbEkt90/FS1N3uj6",
99    "User_Role": "admin"
100  },
101  {
102    "User_ID": 900002,
103    "First_Name": "Kittapon",
104    "Last_Name": "Lerdpayub",
105    "Email": "kittapon.ler@student.mahidol.ac.th",
106    "User_Password": "$2b$10$hs/SQNU0tmX9TiL5AstJleV17v03xMg1Yf80k/YfBwA.d10gU3qzm",
107    "User_Role": "admin"
108  }

```

Movie Manager Service

1. Add new movie

http://localhost:3030/movie/add

Save

POST

http://localhost:3030/movie/add

Send

Params

Authorization

Headers (8)

Body

Pre-request Script

Tests

Settings

Cookies

Beautiful

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

```

1  {
2    "movie": {
3      "Movie_ID": 138989,
4      "Movie_Name": "Turning Red Panda",
5      "Movie_img_src": "https://m.media-amazon.com/images/M/MV5BNjY0MGZmZmQwMmVhLWlANWEtYjQ0MDkyYTJhMDU8XkEyXkFqcGdeQXVyODc0OTEyNDU@.
6      _V1_FMjpg_UX1000_.jpg",
7      "Release_Date": "2012-03-05",
8      "Run_Time": "01:40:00",
9      "Director": "Domee Shi",
10     "Rate": "PG",
11     "Tag": "animation",
12     "Movie_Description": "Mei Lee is a confident, dorky 13-year-old torn between staying her mother's dutiful daughter and the chaos of
13     adolescence. And as if changes to her interests, relationships and body weren't enough, whenever she gets too excited, she poofs into a
    giant red panda"
  }
}

```

Body

Cookies

Headers (8)

Test Results

Status: 200 OK

Time: 7 ms

Size: 344 B

Save Response

Pretty

Raw

Preview

Visualize

JSON

```

1  {
2    "error": false,
3    "data": 1,
4    "message": "New movie has been created successfully."
5  }

```

	Movie_ID	Movie_Name	Movie_img_src	Release_Date	Run_Time	Director	Rate	Tag	Movie_Description
	130001	Avengers: Endgame	Avengers_Endgame.jpg	2019-04-26	03:01:00	Anthony Russo, Joe Russo	PG-13	action	After the devastating events of Avengers: Inf...
	130002	47 Ronin	47 Ronin.jpg	2013-12-06	02:08:00	Carl Rinsch	PG-13	action	A band of samurai set out to avenge the death ...
	130003	The Last Samurai	The Last Samurai.jpg	2019-01-21	02:34:00	Edward Zwick	R	action	An American military advisor embraces the Sam...
	130004	Hotel Transylvania 3: Summer Vacation	Hotel Transylvania 3.jpg	2018-07-13	01:37:00	Genndy Tartakovsky	PG	comedy	Count Dracula and company participate in a cru...
	130005	Zootopia	Zootopia.jpg	2016-04-21	01:48:00	Rich Moore, Byron Howard	PG	comedy	In a city of anthropomorphic animals, a rookie b...
	130006	The silence	The silence.jpg	2019-05-16	01:30:00	John R. Leonetti	PG-16	horror	With the world under attack by deadly creature...
	130007	A WHISKER AWAY	A WHISKER AWAY.jpg	2020-06-18	01:45:00	Junichi Sato, Tomotaka Shibayama	PG	fantasy	A peculiar girl transforms into a cat to catch her...
	130008	Godzilla: King of the Monsters	godzilla_kingmons.jpg	2019-05-31	02:11:00	Michael Dougherty	PG-13	sci-fi	When rogue scientists set out to reset the bala...
	130009	Ghost Wife	Ghost Wife.jpg	2018-11-22	01:40:00	Mate Yimsomboon	PG-16	horror	After being separated from the boy she loves, ...
	130010	Pee Mak	Pee Mak.jpg	2013-03-28	01:55:00	Banjong Pisanthanakun	PG-16	horror, comedy	Mak's friends just want to protect him, but his w...
	130011	2012	2012.jpg	2009-11-13	02:38:00	Roland Emmerich	PG-13	sci-fi	When a flood of natural disasters begins to des...
	130012	The Platform	The Platform.jpg	2019-11-08	01:34:00	Galder Gaztelu-Urrutia	PG-18	horror	A slab of food descends floor by floor in a priso...
	130013	13 Hours: The Secret Soldiers of Beng...	13hrs.jpg	2016-01-21	02:25:00	Michael Bay	PG-18	action	Members of an elite security team battle to sav...
	130014	Deep Impact	Deep Impact.jpg	1998-05-08	02:02:00	Mimi Leder	PG-13	sci-fi	With a massive comet threatening to destroy E...
	130015	Teenage Mutant Ninja Turtles	ninja_turtles.jpg	2014-08-06	01:42:00	Jonathan Liebesman	PG-13	action	With New York City in the criminal clutches of th...
	138989	Turning Red Panda	https://m.media-amazo...	2012-03-05	01:40:00	Domee Shi	PG	animation	Mei Lee is a confident, dorky 13-year-old torn b...

2. Update movie information

http://localhost:3030/movie/update Save

PUT ▼ http://localhost:3030/movie/update Send ▼

Params Authorization Headers (8) **Body** Pre-request Script Tests Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL ☒ JSON ▼ Beautify

```

1  {
2    "movie": {
3      "Movie_ID": 138989,
4      "Movie_Name": "Turning Red",
5      "Movie_img_src": "https://m.media-amazon.com/images/M/MV5BNjY0MGZmZmQtZWxkLWVhLWI4NWVjQ0MDkyYTJhMDU4XkE5XkFqcGdeQXVyODc0OTEyNDU@._V1_FMjpg_UX1000_.jpg",
6      "Release_Date": "2022-03-10",
7      "Run_Time": "01:40:00",
8      "Director": "Domee Shi",
9      "Rate": "PG",
10     "Tag": "animation",
11     "Movie_Description": "Mei Lee is a confident, dorky 13-year-old torn between staying her mother's dutiful daughter and the chaos of adolescence. And as if changes to her interests, relationships and body weren't enough, whenever she gets too excited, she poofs into a giant red panda."
12   }
13 }

```

Body Cookies Headers (8) Test Results Status: 200 OK Time: 6 ms Size: 345 B Save Response ▼

Pretty Raw Preview Visualize JSON ▼

```

1  {
2    "error": false,
3    "data": 1,
4    "message": "This movie has been updated successfully."
5  }

```

Movie_ID	Movie_Name	Movie_img_src	Release_Date	Run_Time	Director	Rate	Tag	Movie_Description
130001	Avengers: Endgame	Avengers_Endgame.jpg	2019-04-26	03:01:00	Anthony Russo, Joe Russo	PG-13	action	After the devastating events of Avengers: Inf...
130002	47 Ronin	47 Ronin.jpg	2013-12-06	02:08:00	Carl Rinsch	PG-13	action	A band of samurai set out to avenge the death ...
130003	The Last Samurai	The Last Samurai.jpg	2019-01-21	02:34:00	Edward Zwick	R	action	An American military advisor embraces the Sam...
130004	Hotel Transylvania 3: Summer Vacation	Hotel Transylvania 3.jpg	2018-07-13	01:37:00	Genndy Tartakovsky	PG	comedy	Count Dracula and company participate in a crui...
130005	Zootopia	Zootopia.jpg	2016-04-21	01:48:00	Rich Moore, Byron Howard	PG	comedy	In a city of anthropomorphic animals, a rookie b...
130006	The silence	The silence.jpg	2019-05-16	01:30:00	John R. Leonetti	PG-16	horror	With the world under attack by deadly creature...
130007	A WHISKER AWAY	A WHISKER AWAY.jpg	2020-06-18	01:45:00	Junichi Sato, Tomotaka Shibayama	PG	fantasy	A peculiar girl transforms into a cat to catch her...
130008	Godzilla: King of the Monsters	godzilla_kingmons.jpg	2019-05-31	02:11:00	Michael Dougherty	PG-13	sci-fi	When rogue scientists set out to reset the bala...
130009	Ghost Wife	Ghost Wife.jpg	2018-11-22	01:40:00	Mate Yimsomboon	PG-16	horror	After being separated from the boy she loves, ...
130010	Pee Mak	Pee Mak.jpg	2013-03-28	01:55:00	Banjong Pisanthanakun	PG-16	horror, comedy	Mak's friends just want to protect him, but his w...
130011	2012	2012.jpg	2009-11-13	02:38:00	Roland Emmerich	PG-13	sci-fi	When a flood of natural disasters begins to des...
130012	The Platform	The Platform.jpg	2019-11-08	01:34:00	Galder Gaztelu-Urrutia	PG-18	horror	A slab of food descends floor by floor in a priso...
130013	13 Hours: The Secret Soldiers of Beng...	13hrs.jpg	2016-01-21	02:25:00	Michael Bay	PG-18	action	Members of an elite security team battle to sav...
130014	Deep Impact	Deep Impact.jpg	1998-05-08	02:02:00	Mimi Leder	PG-13	sci-fi	With a massive comet threatening to destroy E...
130015	Teenage Mutant Ninja Turtles	ninja_turtles.jpg	2014-08-06	01:42:00	Jonathan Liebesman	PG-13	action	With New York City in the criminal clutches of th...
138989	Turning Red	https://m.media-amazo...	2022-03-10	01:40:00	Domee Shi	PG	animation	Mei Lee is a confident, dorky 13-year-old torn b...

3. Delete a movie

http://localhost:3030/movie/delete

DELETE http://localhost:3030/movie/delete

Params Authorization Headers (8) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

1
2
3

```

1  {
2    "Movie_ID": 138989
3  }

```

Body Cookies Headers (8) Test Results

Status: 200 OK Time: 11 ms Size: 345 B Save Response

Pretty Raw Preview Visualize JSON

```

1  {
2    "error": false,
3    "data": 1,
4    "message": "This movie has been deleted successfully."
5  }

```

4. Select a movie by movie ID

http://localhost:3030/movie/select/id/130001

GET http://localhost:3030/movie/select/id/130001

Params Authorization Headers (8) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

1
2
3
4

```

1  {
2    "email": "paveena@student.mahidol.ac.th",
3    "password": "mko098"
4  }

```

Body Cookies Headers (8) Test Results

Status: 200 OK Time: 13 ms Size: 797 B Save Response

Pretty Raw Preview Visualize JSON

```

1  {
2    "error": false,
3    "data": [
4      {
5        "Movie_ID": 130001,
6        "Movie_Name": "Avengers: Endgame",
7        "Movie_img_src": "Avengers_Endgame.jpg",
8        "Release_Date": "2019-04-25T17:00:00.000Z",
9        "Run_Time": "03:01:00",
10       "Director": "Anthony Russo, Joe Russo",
11       "Rate": "PG-13",
12       "Tag": "action",
13       "Movie_Description": "After the devastating events of Avengers: Infinity War (2018), the universe is in ruins. With the help of remaining allies, the Avengers assemble once more in order to reverse Thanos' actions and restore balance to the universe."
14     }
15   ],
16   "message": "Movie retrieved."
17 }

```

5. Select a movie by movie name

http://localhost:3030/movie/select/name/avengers

GET http://localhost:3030/movie/select/name/avengers

Params Authorization Headers (8) Body ● Pre-request Script Tests Settings Cookies

Query Params

KEY	VALUE	DESCRIPTION	...	Bulk Edit
Key	Value	Description		

Body Cookies Headers (8) Test Results Status: 200 OK Time: 5 ms Size: 797 B Save Response

Pretty Raw Preview Visualize JSON

```

1  {
2    "error": false,
3    "data": [
4      {
5        "Movie_ID": 130001,
6        "Movie_Name": "Avengers: Endgame",
7        "Movie_img_src": "Avengers_Endgame.jpg",
8        "Release_Date": "2019-04-25T17:00:00.000Z",
9        "Run_Time": "03:01:00",
10       "Director": "Anthony Russo, Joe Russo",
11       "Rate": "PG-13",
12       "Tag": "action",
13       "Movie_Description": "After the devastating events of Avengers: Infinity War (2018), the universe is in ruins. With the help of remaining allies, the Avengers assemble once more in order to reverse Thanos' actions and restore balance to the universe."
14     }
15   ],
16   "message": "Movie retrieved."
17 }

```

6. Select a movie by movie genre

http://localhost:3030/movie/select/tag/action

GET http://localhost:3030/movie/select/tag/action

Params Authorization Headers (8) Body ● Pre-request Script Tests Settings Cookies

Body Cookies Headers (8) Test Results Status: 200 OK Time: 14 ms Size: 2.2 KB Save Response

Pretty Raw Preview Visualize JSON

```

1  {
2    "error": false,
3    "data": [
4      {
5        "Movie_ID": 130001,
6        "Movie_Name": "Avengers: Endgame",
7        "Movie_img_src": "Avengers_Endgame.jpg",
8        "Release_Date": "2019-04-25T17:00:00.000Z",
9        "Run_Time": "03:01:00",
10       "Director": "Anthony Russo, Joe Russo",
11       "Rate": "PG-13",
12       "Tag": "action",
13       "Movie_Description": "After the devastating events of Avengers: Infinity War (2018), the universe is in ruins. With the help of remaining allies, the Avengers assemble once more in order to reverse Thanos' actions and restore balance to the universe."
14     },
15     {
16       "Movie_ID": 130002,
17       "Movie_Name": "47 Ronin",
18       "Movie_img_src": "47 Ronin.jpg",
19       "Release_Date": "2013-12-05T17:00:00.000Z",
20       "Run_Time": "02:08:00",
21       "Director": "Carl Rinsch",
22       "Rate": "PG-13",
23       "Tag": "action",
24       "Movie_Description": "A band of samurai set out to avenge the death and dishonor of their master at the hands of a ruthless shogun."
25     }
26   ],
27   "message": "Movies retrieved."
28 }

```


7. Select all movies

http://localhost:3030/movie/selectall

GET http://localhost:3030/movie/selectall

Params Authorization Headers (8) Body ● Pre-request Script Tests Settings Cookies

Body Cookies Headers (8) Test Results

Status: 200 OK Time: 8 ms Size: 5.92 KB Save Response

Pretty Raw Preview Visualize JSON

```

1  {
2    "error": false,
3    "data": [
4      {
5        "Movie_ID": 130001,
6        "Movie_Name": "Avengers: Endgame",
7        "Movie_img_src": "Avengers_Endgame.jpg",
8        "Release_Date": "2019-04-25T17:00:00.000Z",
9        "Run_Time": "03:01:00",
10       "Director": "Anthony Russo, Joe Russo",
11       "Rate": "PG-13",
12       "Tag": "action",
13       "Movie_Description": "After the devastating events of Avengers: Infinity War (2018), the universe is in ruins. With the help of remaining
14         allies, the Avengers assemble once more in order to reverse Thanos' actions and restore balance to the universe."
15     },
16     {
17       "Movie_ID": 130002,
18       "Movie_Name": "47 Ronin",
19       "Movie_img_src": "47 Ronin.jpg",
20       "Release_Date": "2013-12-05T17:00:00.000Z",
21       "Run_Time": "02:08:00",
22       "Director": "Carl Rinsch",
23       "Rate": "PG-13",
24       "Tag": "action",
25       "Movie_Description": "A band of samurai set out to avenge the death and dishonor of their master at the hands of a ruthless shogun."
26     }
27   ]
28 }

```